

Last updated on **May 1, 2026**

What is Lakeflow Connect?

Lakeflow Connect offers simple and efficient connectors to ingest data from local files, popular enterprise applications, databases, cloud storage, message buses, and more. This page outlines some of the ways that Lakeflow Connect can improve ETL performance. It also covers common use cases and the range of supported ingestion tools, from fully-managed connectors to fully-customizable frameworks.

Flexible service models

Lakeflow Connect offers a broad range of connectors for enterprise applications, cloud storage, databases, message buses, and more. It also gives you the flexibility to choose between the following:

Option	Description
A fully-managed service	Out-of-the-box connectors that democratize data access with simple UIs and powerful APIs. This allows you to quickly create robust ingestion pipelines while minimizing long-term maintenance costs.
A custom pipeline	If you need more customization, you can use Lakeflow Spark Declarative Pipelines or Structured Streaming. Ultimately, this versatility enables Lakeflow Connect to meet your organization's specific needs.

Unification with core Databricks tools

Lakeflow Connect uses core Databricks features to provide comprehensive data management. For example, it offers governance using Unity Catalog, orcl

 Ask Genie

using Lakeflow Jobs, and holistic monitoring across your pipelines. This helps your organization manage data security, quality, and cost while unifying your ingestion processes with your other data engineering tools. Lakeflow Connect is built on an open Data Intelligence Platform, with full flexibility to incorporate your preferred third-party tools. This ensures a tailored solution that aligns with your existing infrastructure and future data strategies.

Fast, scalable ingestion

Lakeflow Connect uses incremental reads and writes to enable efficient ingestion. When combined with [incremental transformations downstream](#), this can significantly improve ETL performance.

Common use cases

Customers ingest data to solve their organizations' most challenging problems. Sample use cases include the following:

Use case	Description
Customer 360	Measuring campaign performance and customer lead scoring
Portfolio management	Maximizing ROI with historical and forecasting models
Consumer analytics	Personalizing your customers' purchasing experiences
Centralized human resources	Supporting your organization's workforce
Digital twins	Increasing manufacturing efficiency
RAG chatbots	Building chatbots to help users understand policies, products, and more

Layers of the ETL stack

Some connectors operate at one level of the ETL stack. For example, Databricks offers fully-managed connectors for enterprise applications like Salesforce and databases like SQL Server. Other connectors operate at a different layer of the ETL stack. For example, you can use standard connectors in Lakeflow Spark Declarative Pipelines for more customization options. Similarly, you can choose your level of customization for streaming data from Apache Kafka, Amazon Kinesis, Google Pub/Sub, and Apache Pulsar.

Databricks recommends starting with the most managed layer. If it doesn't satisfy your requirements (for example, if it doesn't support your data source), drop down to the next layer.

The following table describes the layers of ingestion products:

Layer	Description
Lakeflow Spark Declarative Pipelines	Lakeflow Spark Declarative Pipelines offers a declarative framework for creating data pipelines. Define your transformations, and Lakeflow Spark Declarative Pipelines manages orchestration, monitoring, data quality, errors, and more. It builds on Structured Streaming for streaming and supports most Structured Streaming features. For any Structured Streaming feature not yet available in Lakeflow Spark Declarative Pipelines, you can use Structured Streaming APIs directly.
Fully-managed connectors	Fully-managed connectors build on Lakeflow Spark Declarative Pipelines, offering even more automation for the most popular data sources. They extend Lakeflow Spark Declarative Pipelines functionality to also include source-specific authentication, CDC, edge case handling, long-term API maintenance, automated retries, automated schema evolution, and so on. Therefore, they offer even more automation for any supported data sources.

Managed connectors

You can use fully-managed connectors to ingest from enterprise applications and databases. See [Managed connectors in Lakeflow Connect](#) for a full list of supported connectors.

Supported interfaces include:

- Databricks UI
- Declarative Automation Bundles
- Databricks APIs
- Databricks SDKs
- Databricks CLI

Community connectors

Community connectors extend Lakeflow Connect to sources without managed connector support. They're built and maintained by the community and are not backed by Databricks SLAs. You can use an existing connector or create your own. See [Community connectors in Lakeflow Connect](#).

Standard connectors

In addition to the managed connectors, Databricks offers customizable connectors for cloud object storage and message buses. See [Standard connectors in Lakeflow Connect](#).

Create or modify a table from file upload (Add data UI)

You can ingest files that reside on your local network, files uploaded to a volume, or files downloaded from an internet location. See [Create or modify a table using file upload](#).

Ingestion partners

Many third-party tools support batch or streaming ingestion into Databricks. Databricks validates various third-party integrations, although the steps to configure access to source systems and ingest data vary by tool. See [Ingestion partners](#) for a list of validated tools. Some technology partners are also featured in [Databricks Partner Connect](#), which has a UI that simplifies connecting third-party tools to Lakehouse data.

DIY ingestion

Databricks provides a general compute platform. As a result, you can create your own ingestion connectors using any programming language supported by Databricks, like Python or Java. You can also import and use popular open source connector libraries like data load tool, Airbyte, and Debezium.

Ingestion alternatives

Databricks recommends ingestion for most use cases because it scales to accommodate high data volumes, low-latency querying, and third-party API limits. Ingestion copies data from your source systems to Databricks, which results in duplicate data that might become stale over time. If you don't want to copy data, you can use the following tools:

Tool	Description
Lakehouse Federation	Allows you to query external data sources without moving your data.
Delta Sharing	Allows you to securely share data across platforms, clouds, and regions.

Last updated on **Mar 31, 2026**

Create or modify a table using file upload

The **Create or modify a table using file upload** page allows you to upload CSV, TSV, or JSON, Avro, Parquet, or text files to create or overwrite a managed Delta Lake table.

You can create managed Delta tables in Unity Catalog or in the Hive metastore.

NOTE

In addition, you can use [use the add data UI](#) or [COPY INTO](#) to load files from cloud storage.

IMPORTANT

- You must have access to a running compute resource and permissions to create tables in a target schema.
- Workspace admins can [disable the Create or modify a table using file upload page](#).

You can use the UI to create a Delta table by importing small CSV, TSV, JSON, Avro, Parquet, or text files from your local machine.

- The **Create or modify a table using file upload** page supports uploading up to 10 files at a time.
- The total size of uploaded files must be under 2 gigabytes.
- The file must be a CSV, TSV, JSON, Avro, Parquet, or text file and have the extension ".csv", ".tsv" (or ".tab"), ".json", ".avro", ".parquet", or ".txt".
- Compressed files such as `zip` and `tar` files are not supported.

Upload the file

 Ask Genie

1. Click  **New > Add or upload data.**
2. Click **Create or modify a table.**
3. Click **browse** or drag and drop files directly on the drop zone.

 NOTE

Imported files are uploaded to a secure internal location within your account which is garbage collected daily.

Preview, configure, and create a table

You can upload data to the staging area without connecting to compute resources, but you must select an active compute resource to preview and configure your table. The file upload UI supports SQL warehouses, serverless compute, and dedicated compute. Group clusters are not supported.

You can preview 50 rows of your data when you configure the options for the uploaded table. Click the grid or list buttons under the file name to switch the presentation of your data.

Databricks stores data files for managed tables in the locations configured for the containing schema. You need proper permissions to create a table in a schema.

Select the desired schema in which to create a table by doing the following:

1. (For Unity Catalog-enabled workspaces only) You can select a catalog or the legacy `hive_metastore`.
2. Select a schema.
3. (Optional) Edit the table name.

 NOTE

You can use the dropdown to select **Overwrite existing table** or **Create new table**. Operations that attempt to create new tables with name conflicts display an error message.

You can configure [options](#) or [columns](#) before you create the table.

 Ask Genie

To create the table, click **Create** at the bottom of the page.

Format options

Format options depend on the file format you upload. Common format options appear in the header bar, while less commonly used options are available on the **Advanced attributes** dialog.

- For CSV, the following options are available:
 - **First row contains the header** (enabled by default): This option specifies whether the CSV/TSV file contains a header.
 - **Column delimiter**: The separator character between columns. Only a single character is allowed, and backslash is not supported. This defaults to comma for CSV files.
 - **Automatically detect column types** (enabled by default): Automatically detect column types from file content. You can edit types in the preview table. If this is set to false, all column types are inferred as `STRING`.
 - **Rows span multiple lines** (disabled by default): Whether a column's value can span multiple lines in the file.
 - **Merge the schema across multiple files**: Whether to infer the schema across multiple files and to merge the schema of each file. If disabled, the schema from one file is used.
- For JSON, the following options are available:
 - **Automatically detect column types** (enabled by default): Automatically detect column types from file content. You can edit types in the preview table. If this is set to false, all column types are inferred as `STRING`.
 - **Rows span multiple lines** (enabled by default): Whether a column's value can span multiple lines in the file.
 - **Allow comments** (enabled by default): Whether comments are allowed in the file.
 - **Allow single quotes** (enabled by default): Whether single quotes are allowed in the file.
 - **Infer timestamp** (enabled by default): Whether to try to infer timestamp strings as `TimestampType`.

 Ask Genie

The data preview updates automatically when you edit format options.

NOTE

When you upload multiple files, the following rules apply:

- Header settings apply to all files. Make sure headers are consistently absent or present in all uploaded files to avoid data loss.
- Uploaded files combine by appending all data as rows in the target table. Joining or merging records during file upload is not supported.

Column names and types

You can edit column names and types.

- To edit types, click the icon with the type.

NOTE

You can't edit nested types for `STRUCT` or `ARRAY`.

- To edit the column name, click the input box at the top of the column.

Column names do not support commas, backslashes, or unicode characters (such as emojis).

Column data types are inferred by default for CSV and JSON files. You can interpret all columns as `STRING` type by disabling **Advanced attributes** > **Automatically detect column types**.

NOTE

- Schema inference does a best effort detection of column types. Changing column types can lead to some values being cast to `NULL` if the value cannot be cast correctly to the target data type. Casting `BIGINT` to `DATE` or `TIMESTAMP` columns is not supported. Databricks recommends that you create a table first and then transform these columns using SQL functions afterwards.
- To support table column names with special characters, the **Create or modify a table using file upload** page leverages [Column Mapping](#).  Ask Genie

- To add comments to columns, create the table and navigate to [Catalog Explorer](#) where you can add comments.

Supported data types

The **Create or modify a table using file upload** page supports the following data types. For more information about individual data types see [SQL data types](#).

Data Type	Description
<code>BIGINT</code>	8-byte signed integer numbers.
<code>BOOLEAN</code>	Boolean (<code>true</code> , <code>false</code>) values.
<code>DATE</code>	Values comprising values of fields year, month, and day, without a time-zone.
<code>DOUBLE</code>	8-byte double-precision floating point numbers.
<code>STRING</code>	Character string values.
<code>TIMESTAMP</code>	Values comprising values of fields year, month, day, hour, minute, and second, with the session local timezone.
<code>STRUCT</code>	Values with the structure described by a sequence of fields.
<code>ARRAY</code>	Values comprising a sequence of elements with the type <code>elementType</code> .
<code>DECIMAL(P,S)</code>	Numbers with maximum precision <code>P</code> and fixed scale <code>S</code> .

Known issues

Casting `BIGINT` to non-castable types like `DATE`, such as dates in the format of 'yyyy', may trigger errors.

 Ask Genie

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **May 1, 2026**

Managed connectors in Lakeflow Connect

📘 NOTE

Managed connectors in Lakeflow Connect are in various [release states](#).

This page provides an overview of managed connectors in Databricks Lakeflow Connect for ingesting data from SaaS applications and databases. The resulting ingestion pipeline is governed by Unity Catalog and is powered by serverless compute and Lakeflow Spark Declarative Pipelines. Managed connectors leverage efficient incremental reads and writes to make data ingestion faster, scalable, and more cost-efficient, while your data remains fresh for downstream consumption.

Connector types

SaaS connectors

Ingest data from enterprise SaaS applications including Salesforce, HubSpot, Jira, Workday, and more.

Database connectors (CDC)

Ingest data from relational databases including MySQL, PostgreSQL, and SQL Server using change data capture.

Community connectors

Ingest data using open-source, community-built connectors.

Architecture

Each connector type has a distinct set of components. SaaS connectors use a connection, an ingestion pipeline, and destination tables. Database connectors also include an ingestion gateway and staging storage to support continuous change capture. For details, see [SaaS connectors in Lakeflow Connect](#) and [Database connectors in Lakeflow Connect](#).

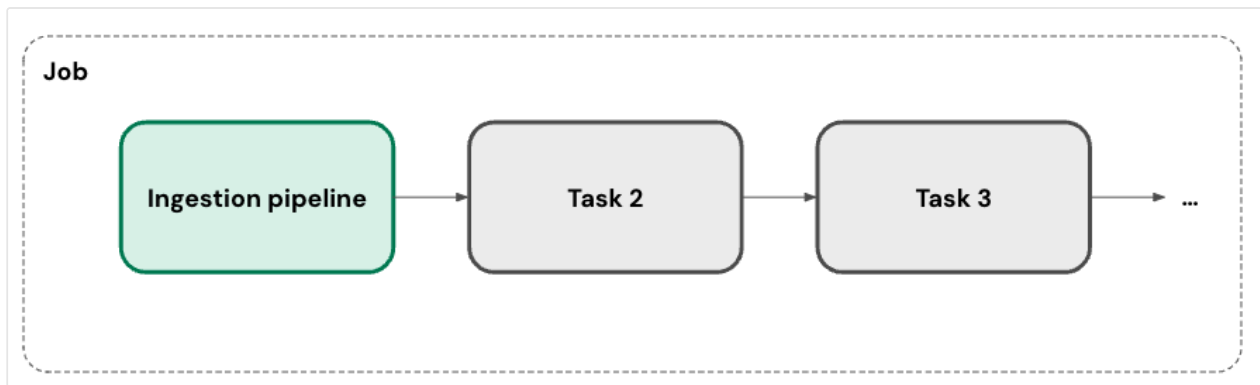
Query-based connector components

A query-based connector queries the source database directly on a schedule, without a gateway or staging storage. For an overview of how query-based connectors work, see [Query-based connectors](#).

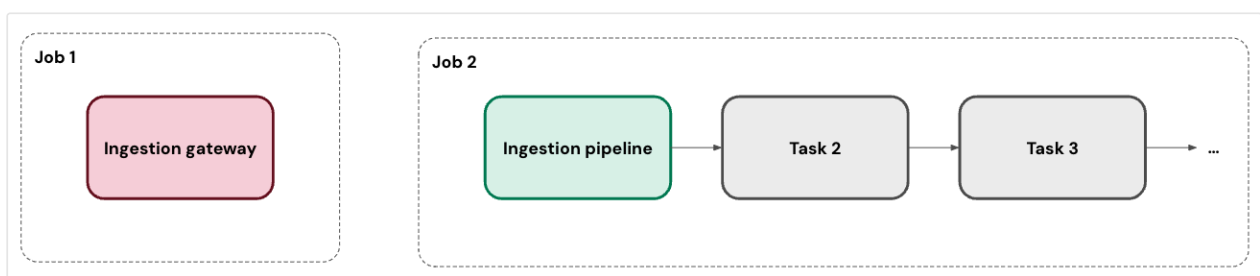
Component	Description
Connection	A Unity Catalog securable object that stores authentication details for the source database. Either a direct Unity Catalog connection (for foreign connection ingestion) or a Unity Catalog foreign catalog (for foreign catalog ingestion using Lakehouse Federation).
Ingestion pipeline	A pipeline that queries the source database directly and writes the results to streaming tables. The pipeline runs on serverless compute by default.
Destination tables	The streaming tables where the ingestion pipeline writes the data.

Orchestration

You can run your ingestion pipeline on one or more custom schedules. For each schedule that you add to a pipeline, Lakeflow Connect automatically creates a [job](#) for it. The ingestion pipeline is a task within the job. You can optionally add more tasks to the job.



For database connectors, the ingestion gateway runs in its own job as a continuous task.



Incremental ingestion

Lakeflow Connect uses incremental ingestion to improve pipeline efficiency. On the first run of your pipeline, it ingests all of the selected data from the source. In parallel, it tracks changes to the source data. On each subsequent run of the pipeline, it uses that change tracking to ingest only the data that's changed from the prior run, when possible.

The exact approach depends on what's available in your data source. For example, you can use both change tracking and change data capture (CDC) with SQL Server. In contrast, the Salesforce connector selects a cursor column from a set list of options.

Some sources or specific tables don't support incremental ingestion at this time. Databricks plans to expand coverage for incremental support.

Networking

There are several options for connecting to a SaaS application or database.

- Connectors for SaaS applications reach out to the source's APIs. They're also automatically compatible with serverless egress controls.
- Connectors for cloud databases can connect to the source via Private Link. Alternatively, if your workspace has a Virtual Network (VNet) or Virtual Private Cloud (VPC) that's peered with the VNet or VPC hosting your database, then you can deploy the gateway inside of it.
- Connectors for on-premises databases can connect using services like AWS Direct Connect and Azure ExpressRoute.

Deployment

You can deploy ingestion pipelines using [Declarative Automation Bundles](#), which enable best practices like source control, code review, testing, and continuous integration and delivery (CI/CD). Bundles are managed using the Databricks CLI and can be run in different target workspaces, such as development, staging, and production.

Failure recovery

As a fully-managed service, Lakeflow Connect aims to automatically recover from issues when possible. For example, when a connector fails, it automatically retries with exponential backoff.

However, it's possible that an error requires your intervention (for example, when credentials expire). In these cases, the connector tries to avoid missing data by storing the last position of the cursor. It can then pick back up from that position on the next run of the pipeline when possible.

Monitoring

Lakeflow Connect provides robust alerting and monitoring to help you maintain your pipelines. This includes event logs, cluster logs, pipeline health metrics, and data quality metrics. You can also use the `system.billing.usage` table to track costs and monitor pipeline usage. See [Monitor managed ingestion pipeline cost](#).  Ask Genie

For database connectors, you can monitor gateway progress in real time using event logs. See [Monitor ingestion gateway progress with event logs](#).

Community connectors

Community connectors extend Lakeflow Connect to sources without managed connector support. They're built and maintained by the community. You can use an existing connector or create your own. See [Community connectors in Lakeflow Connect](#).

Dependence on external services

Databricks SaaS, database, and other fully-managed connectors depend on the accessibility, compatibility, and stability of the application, database, or external service they connect to. Databricks does not control these external services and, therefore, has limited (if any) influence over their changes, updates, and maintenance.

If changes, disruptions, or circumstances related to an external service impede or render impractical the operation of a connector, Databricks may discontinue or cease maintaining that connector. Databricks will make reasonable efforts to notify customers of discontinuation or cessation of maintenance, including updates to the applicable documentation.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **May 8, 2026**

Common patterns for managed ingestion pipelines

Lakeflow Connect provides patterns and techniques to optimize your managed ingestion pipelines. Use these patterns to control which data is ingested, manage pipeline updates, and configure advanced behaviors.

Not all connectors support the common patterns in this section.

Topic	Description
Column selection	Select or exclude specific columns during ingestion to reduce data volume and improve performance.
Full refresh	Force a complete reload of your data from the source system.
History tracking	Track historical changes in your data using Slowly Changing Dimension (SCD) type 2.
Monitor costs	Use system tables to track pipeline costs and monitor usage patterns.
Multi-destination pipelines	Ingest data from a single source to multiple destination tables or catalogs.
Pipeline maintenance	Manage pipeline updates, pauses, and troubleshooting workflows.
 Ask Genie	

Topic	Description
Pipeline tagging	Apply pipeline tags for resource organization, ownership tracking, and cost attribution.
Row filtering	Filter rows during ingestion using SQL-like conditions.
Configure the Run as identity	Configure which identity's permissions are used when the pipeline runs.
Source data lineage	Track source table lineage for ingested data so that source tables appear in Unity Catalog end-to-end lineage views.
Name destination tables	Name destination tables. By default, a destination table is given the name of the corresponding source table. However, naming a destination table is useful when you ingest the same source object twice in the same schema. Managed connectors don't support duplicate table names in the same destination schema. Naming a destination table can also align tables to your organization's naming conventions.
TLS server certificate validation	Configure TLS certificate validation for database connector pipelines to verify the source database server's identity and prevent person-in-the-middle (PITM) attacks.

Was this page helpful?

☒ Yes

☐ No

☐ Send feedback

Last updated on **May 4, 2026**

Managed connector FAQs

Find answers to frequently asked questions about managed connectors in Databricks Lakeflow Connect. For connector-specific FAQs, see the documentation for your connector.

Which managed connectors does Databricks support?

Lakeflow Connect offers managed connectors for Salesforce, SQL Server, ServiceNow, and Google Analytics. To inform the roadmap or gain early access to Private Preview connectors, reach out to your account team.

Which interfaces do managed connectors support?

All managed connectors support pipeline creation using Databricks APIs and Declarative Automation Bundles. Some connectors also support pipeline creation in the UI from the following entry points:

- The **Add data** page (**Data Ingestion**)
- The **Jobs & Pipelines** page (**Create new** > **Ingestion pipeline**)
- The **Tasks** pane for a new job (**Add task** > **+ New ingestion pipeline** or **Type** > **Pipeline**)

For supported interfaces, see the **Feature availability** section on the overview page for your connector.

How do managed connectors handle schema evolution?

All managed connectors automatically handle new and deleted columns unless you opt out by explicitly specifying the columns that you'd like to ingest.

- When a new column appears in the source, Databricks automatically ingests it on the next run of your pipeline. For any row in the column that appeared prior to the schema change, Databricks leaves the value empty. However, you can opt out of automated column ingestion by listing specific columns to ingest via the API or disabling any future columns in the UI.
- When a column is deleted from the source, Databricks doesn't delete it automatically. Instead, the connector uses a table property to set the deleted column to "inactive" in the destination. If another column later appears that has the same name, then the pipeline fails. In this case, you can trigger a full refresh of the table or manually drop the inactive column.

Similarly, connectors can handle new and deleted tables. If you ingest an entire schema, Databricks automatically ingests any new tables, unless you opt out. And if a table is deleted in the source, the connector sets it to `inactive` in the destination. If you choose to ingest an entire schema, review the limitations on the number of tables per pipeline for your connector.

Additional schema changes depend on the source. For example, the Salesforce connector treats column renames as column deletions and additions and automatically makes the change, with the behavior outlined above. However, the SQL Server connector requires a full refresh of the affected tables to continue ingestion.

For more information about schema evolution behavior, see the **Feature availability** section on the overview page for your connector.

Can I customize managed connectors?

You can choose the ingested objects, destination, schedule, permissions, notifications, and more. You can't customize the ingestion process itself because these connectors

are fully managed. For further customization, you can use Lakeflow Spark Declarative Pipelines or Structured Streaming.

What's the difference between managed connectors, Lakehouse Federation, and Delta Sharing?

Lakehouse Federation allows you to query external data sources without moving your data. Delta Sharing allows you to securely share live data across platforms, clouds, and regions.

When you have a choice between managed connectors, Lakehouse Federation, and Delta Sharing, choose Delta Sharing for the following scenarios:

- Limiting data duplication.
- Querying the freshest possible data.

Choose Lakehouse Federation for the following scenarios:

- Ad hoc reporting or proof-of-concept work on your ETL pipelines.

What's the difference between managed connectors and Auto Loader?

Managed connectors allow you to incrementally ingest data from SaaS applications like Salesforce and databases like SQL Server. Auto Loader is a cloud object storage connector that allows you to incrementally ingest files as they arrive in S3, ADLS, and GCS. It is compatible with Structured Streaming and Lakeflow Spark Declarative Pipelines but does not offer fully-managed ingestion pipelines.

Can managed connectors write back to the data source?

No. If you're interested in this functionality, reach out to your account team.

Are managed connectors available in all regions?

No. See [Ingestion availability](#).

Can a pipeline write to multiple destination schemas?

This feature is supported in the Lakeflow Connect API for all managed SaaS connectors, like Salesforce, Workday, and ServiceNow.

If you choose to use this feature, your pipeline will become API-only. You can't edit it in the UI.

Can I change the name of a table that I ingest?

Yes, you can set a name for the destination table in the Databricks UI or using Databricks APIs. See [Name a destination table](#). If you do not set a name, the source table name is used by default.

What happens if a pipeline is still running (update N) when the next update is scheduled to run (update N+1)?

Databricks skips update N+1 and picks up on update N+2, assuming that update N has completed in time.

What happens to the destination tables when an ingestion pipeline is deleted?

The destination tables are dropped when the ingestion pipeline is deleted.

Can I use `ALTER` to modify streaming tables or materialized views created by managed ingestion pipelines?

Yes, you can modify streaming tables and materialized views created by managed ingestion pipelines using the following `ALTER` SQL statements:

SQL

```
ALTER MATERIALIZED VIEW view_name | ALTER STREAMING TABLE table_name
{
    ALTER COLUMN column_clause |
    SET ROW FILTER clause |
    DROP ROW FILTER |
    SET TAGS clause |
    UNSET TAGS clause
}

column_clause
{
    column_identifier
    COMMENT clause |
    SET MASK clause |
    DROP MASK |
    SET TAGS clause |
    UNSET TAGS clause
}
```

For complete syntax and examples, see [ALTER STREAMING TABLE](#) and [ALTER MATERIALIZED VIEW](#).

 **NOTE**

 Ask Genie

You cannot modify the schedule or trigger of a managed ingestion pipeline using an `ALTER` statement. See [Update the pipeline schedule](#).

How are managed connectors priced?

Managed connectors have a compute-based pricing model.

SaaS sources like Salesforce and Workday, which run exclusively on serverless infrastructure, incur serverless Lakeflow Spark Declarative Pipelines DBU charges.

For database sources like SQL Server, ingestion gateways can run in classic mode or serverless mode depending on the source, and ingestion pipelines run on serverless. As a result, you can receive both classic and serverless Lakeflow Spark Declarative Pipelines DBU charges.

For rate details, see the [Lakeflow Spark Declarative Pipelines pricing page](#).

Can an ingestion gateway run in a serverless-only workspace?

No. Gateways require classic compute and can't be deployed in a workspace that doesn't support classic compute. This only applies to database (CDC) connectors.

Can I enable the Delta Lake change data feed on a target table?

This is enabled for all target tables.

How does row filtering handle rows or queries that change after the initial load?

Row filtering behavior varies depending on whether a row or the filter query is updated after the initial load. For a summary of edge case scenarios, see [Row filtering behavior in edge cases](#).

Connector-specific FAQs

For connector-specific FAQs, see the documentation for your connector:

- [Confluence connector FAQs](#)
- [Microsoft Dynamics 365 connector FAQs](#)
- [Google Analytics Raw Data connector FAQs](#)
- [Jira connector FAQs](#)
- [Meta Ads ingestion connector FAQs](#)
- [MySQL connector FAQ](#)
- [PostgreSQL connector FAQs](#)
- [Salesforce ingestion connector FAQs](#)
- [ServiceNow connector FAQs](#)
- [Microsoft SharePoint connector FAQs](#)
- [SQL Server connector FAQs](#)
- [Workday Reports connector FAQs](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 13, 2026**

SaaS connectors in Lakeflow Connect

Databricks Lakeflow Connect provides fully-managed connectors for ingesting data from enterprise SaaS applications. Each connector handles source-specific authentication, incremental reads, schema evolution, and automated retries.

Supported connectors

Confluence

Ingest pages, spaces, and comments from Atlassian Confluence.

Microsoft Dynamics 365

Ingest CRM and ERP data from Microsoft Dynamics 365 using Azure Synapse Link.

Google Ads

Ingest campaign performance and advertising metrics from Google Ads.

Google Analytics

Ingest website traffic and user behavior data from Google Analytics 4.

HubSpot

Ingest CRM, marketing, and sales activity data from HubSpot.

Jira

 Ask Genie

Ingest project management and issue tracking data from Atlassian Jira.

Meta Ads

Ingest advertising campaign data from Meta (Facebook and Instagram Ads).

NetSuite

Ingest ERP and financial data from Oracle NetSuite.

Salesforce

Ingest CRM records and customer data from Salesforce using incremental reads.

ServiceNow

Ingest IT service management records and tickets from ServiceNow.

SharePoint

Ingest files and list data from Microsoft SharePoint sites.

TikTok Ads

Ingest advertising performance data from TikTok Ads.

Workday HCM

Ingest HR, payroll, and workforce data from Workday Human Capital Management.

Workday Reports

Ingest custom Workday reports into Delta tables.

Zendesk Support

Ingest support tickets, users, and organization data from Zendesk.

 **NOTE**

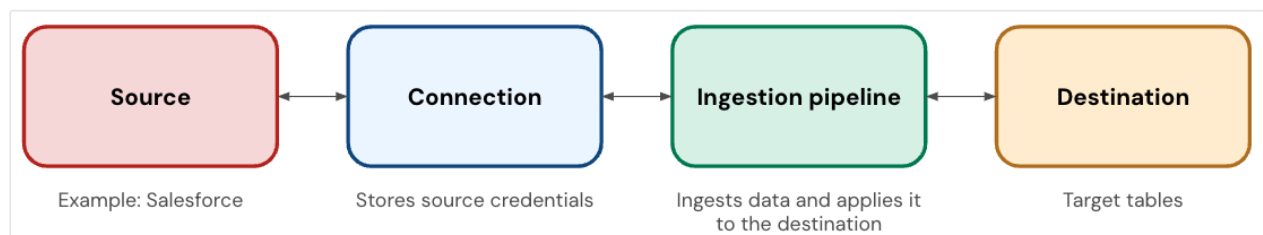
 Ask Genie

To share data with SAP Business Data Cloud (BDC), use the SAP BDC connector for Databricks, which uses Delta Sharing and not Lakeflow Connect. See [Share data between SAP Business Data Cloud \(BDC\) and Databricks](#).

Connector components

A SaaS connector has the following components:

Component	Description
Connection	A Unity Catalog securable object that stores authentication details for the application.
Ingestion pipeline	A pipeline that copies the data from the application into the destination tables. The ingestion pipeline runs on serverless compute.
Destination tables	The tables where the ingestion pipeline writes the data. These are streaming tables , which are Delta tables with extra support for incremental data processing.



Is this page

as this page helpful?

☐ Yes

☐ No

[Ask Genie](#)

☐ Send feedback

Last updated on **May 4, 2026**

Database connectors in Lakeflow Connect

Databricks Lakeflow Connect provides fully-managed connectors for ingesting data from relational databases using change data capture (CDC). Each connector efficiently tracks changes in the source database and applies them incrementally to Delta tables.

Supported connectors

MySQL

Ingest data from MySQL databases using change data capture (CDC) for efficient incremental loads.

PostgreSQL

Ingest data from PostgreSQL databases using change data capture (CDC).

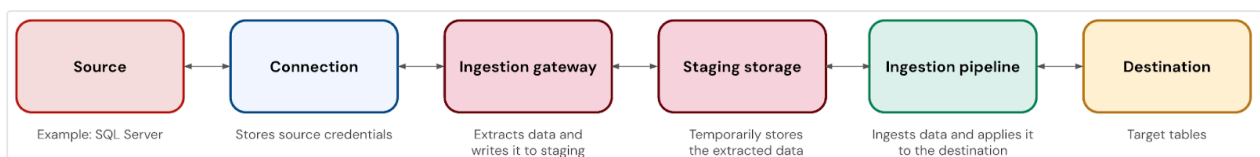
Microsoft SQL Server

Ingest data from Microsoft SQL Server using change data capture (CDC) or full snapshot.

Connector components

A database connector has the following components:

Component	Description
Connection	A Unity Catalog securable object that stores authentication details for the database.
Ingestion gateway	A pipeline that extracts snapshots, change logs, and metadata from the source database. The gateway runs on classic compute, and it runs continuously to capture changes before change logs can be truncated in the source.
Staging storage	A Unity Catalog volume that temporarily stores extracted data before it's applied to the destination table. This allows you to run your ingestion pipeline at whatever schedule you'd like, even as the gateway continuously captures changes. It also helps with failure recovery. You automatically create a staging storage volume when you deploy the gateway, and you can customize the catalog and schema where it lives. Data is automatically purged from staging after 30 days.
Ingestion pipeline	A pipeline that moves the data from staging storage into the destination tables. The pipeline runs on serverless compute.
Destination tables	The tables where the ingestion pipeline writes the data. These are streaming tables , which are Delta tables with extra support for incremental data processing.



Network connectivity

The ingestion gateway runs on classic compute in your Databricks workspace VPC or VNet, and must be able to reach the source database over the network.

Any network path that allows the gateway to reach the database is supported, including VPN, Azure ExpressRoute, AWS Direct Connect, VPC or VNet peering, and public endpoints.

Cross-cloud connectivity is supported. For example, an Azure Databricks workspace can ingest from an AWS Aurora PostgreSQL database if the appropriate network connectivity exists between the two environments.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 15, 2026**

Query-based connectors

ⓘ PREVIEW

This feature is in [Public Preview](#).

Query-based connectors in Lakeflow Connect ingest data from databases by querying the source directly, without requiring change data capture (CDC) configuration. Instead of relying on binlogs or CDC infrastructure, they use a cursor column—a monotonically increasing timestamp or integer column—to track which rows are new or updated since the last pipeline run.

Query-based connectors use Unity Catalog connections and [Lakehouse Federation](#) to connect to source databases, and they write results to [streaming tables](#).

How it works

On each pipeline run, a query-based connector queries the source database and retrieves all rows with a cursor column value that is greater than the value recorded from the previous run. The connector stores the high-water mark of the cursor column after each successful run and uses it as the lower bound on the next run.

Because the connector queries the source directly, it doesn't require an ingestion gateway or a staging volume. The pipeline runs on a schedule you define, not continuously.

Query-based connectors compared to CDC database connectors

Query-based connectors differ from CDC database connectors in the following ways.

- **No ingestion gateway:** CDC connectors require a gateway to capture binlog events. Query-based connectors don't use a gateway.
- **No staging volume:** CDC connectors buffer extracted data in a staging volume. Query-based connectors write directly from the source query to the destination table.
- **Scheduled instead of continuous:** Query-based connectors run on a schedule. They don't capture every intermediate row state between runs. They only capture the latest state of the rows that have changed.
- **Broader source compatibility:** Any database with a suitable cursor column is a valid source, even if it doesn't support CDC or binlog access.

The trade-off is that query performance can be slower and queries run directly on source tables, which can place more load on the source database compared to CDC connectors that query the binlog. Soft-deletion tracking is supported using `deletion_condition`. Hard-deletion tracking is also supported in [Beta](#). Both require API configuration.

Supported ingestion approaches

Query-based connectors support multiple ingestion approaches. The approach you use determines which configuration parameters are required.

Approach	How it connects	Required parameters
Foreign connection ingestion	Uses a connection that stores authentication credentials for the source database. The connector uses the connection to query the source database directly.	<code>connection_name</code> , <code>source_catalog</code> , <code>source_schema</code> , <code>source_table</code> , <code>cursor_column</code>
Foreign catalog ingestion	Uses a foreign catalog backed by a Lakehouse Federation data source.	<code>ingest_from_uc_foreign_catalog:</code> <code>true</code> , <code>cursor_columns</code> , <code>primary_keys</code>  Ask Genie

Approach	How it connects	Required parameters
	The connector uses the foreign catalog to read source data instead of connecting to the source database directly.	(required unless you use <code>APPEND_ONLY</code> mode)

Supported sources

The following database sources are supported.

Foreign connection ingestion sources:

- Oracle
- Teradata
- SQL Server
- MySQL
- MariaDB
- PostgreSQL

Foreign catalog ingestion sources:

All Lakehouse Federation data sources are supported using foreign catalog ingestion. For the full list, see [Lakehouse Federation](#).

Supported interfaces

You can use the Databricks UI or Declarative Automation Bundles to create query-based pipelines.

Compute requirements

Query-based ingestion pipelines run on serverless compute by default. Classic compute is supported in [Beta](#), but only using APIs. Databricks recommends using serverless compute.

To use query-based connectors with serverless compute, your compute environment must allow network connectivity to the source database. See [Networking](#) and [Networking recommendations for Lakehouse Federation](#).

History tracking (SCD) modes

Query-based connectors support the following history tracking modes—also known as slowly changing dimension (SCD) modes—for destination tables:

- **SCD_TYPE_1:** Overwrites the existing row in the destination table with the latest source row. No history is preserved.
- **SCD_TYPE_2:** Preserves the full history of row changes by adding new rows with version metadata. See [Enable history tracking \(SCD type 2\)](#).
- **APPEND_ONLY:** Appends every ingested row to the destination table without merging or overwriting.

Schema evolution

Query-based connectors handle schema evolution the same way as other managed connectors in Lakeflow Connect. See [How do managed connectors handle schema evolution?](#)

Is this page

as this page helpful?

☐ Yes

☐ No

 Ask Genie

Last updated on **May 1, 2026**

Community connectors in Lakeflow Connect

⚠️ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

Community connectors are open-source connectors that extend Lakeflow Connect to sources without managed connector support. The community builds and maintains them. You have the following options:

Use a registered community connector

Use a registered community connector to ingest data from a supported source into Databricks.

Create a custom connector

Create a custom connector for a new source using the community framework and templates.

How community connectors work

Community connectors are built on the `LakeflowConnect` interface, which wraps the Spark Python Data Source API. Each connector handles authentication, schema discovery, and incremental data reads so you can create, configure, and run an ingestion pipeline backed by [Lakeflow Spark Declarative Pipelines](#).

When you use a community connector, Databricks clones the connector source code from a GitHub repository into a workspace directory that you specify. The pipeline then reads the connector source code at runtime and executes the ingestion against the configured source.

 Ask Genie

Supported sources

The community regularly registers new connectors. For the latest list of supported sources, see the Add data UI (**Data Ingestion**) in your Databricks workspace or the [Lakeflow Community Connectors](#) repository on GitHub.

Considerations

- Community connectors are under active development. Interfaces and behavior are subject to change.
- Databricks doesn't maintain community connectors. They're not backed by Databricks SLAs and don't guarantee forward compatibility.

Submit feedback

Report bugs and submit feature requests in the [Lakeflow Community Connectors](#) repository.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 1, 2026**

Use a registered community connector

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

This page shows how to use a registered community connector to ingest data from a supported source into Databricks. To create a custom connector for a source that isn't supported yet, see [Create a custom connector](#).

Requirements

- A Databricks workspace with Unity Catalog enabled
- A connection for the source you want to ingest, or permissions to create a connection
- Write access to a catalog and schema for the ingested tables

Create an ingestion pipeline

To use a registered community connector:

1. In the sidebar of your Databricks workspace, click **+New** > **Add or upload data**, then select the source under **Community connectors**.
2. Click **+ Create connection** or select an existing connection, then click **Next**.
3. For **Pipeline name**, enter a name for the pipeline.

 Ask Genie

4. For **Event log location**, enter a catalog name and a schema name. Databricks stores the pipeline event log here. Ingested tables are also written here by default.
5. For **Root path**, enter your workspace path (for example, `/Workspace/Users/<your-email>/connectors`). Databricks clones and stores the connector source code here.
6. Click **Create pipeline**.
7. In the pipeline editor, open `ingest.py` and update the **objects** field to include the tables you want to ingest. For example:

Python

```
from databricks.labs.community_connector.pipeline import ingest

pipeline_spec = {
    "connection_name": "my_stripe_connection", # Required: UC
    "objects": [
        {"table": {"source_table": "charges"}},
        {"table": {"source_table": "customers",
                    "destination_table": "stripe_customers"}},
    ],
}

ingest(spark, pipeline_spec)
```

8. Run the pipeline manually or schedule it.

Pipeline configuration options

You can configure the following options in `ingest.py`:

Option	Description
<code>connection_name</code>	Required. The name of the connection that stores authentication credentials for the source.
 Ask Genie	

Option	Description
<code>objects</code>	Required. A list of tables to ingest. Each entry has the format <code>{"table": {"source_table": "..."} }</code> . You can also specify an optional <code>destination_table</code> inside the <code>table</code> object.
<code>destination_catalog</code>	The catalog where ingested tables are written. Defaults to the catalog set during pipeline creation.
<code>destination_schema</code>	The schema where ingested tables are written. Defaults to the schema set during pipeline creation.
<code>scd_type</code>	The slowly changing dimension strategy: <code>SCD_TYPE_1</code> , <code>SCD_TYPE_2</code> , or <code>APPEND_ONLY</code> . Defaults to <code>SCD_TYPE_1</code> .
<code>primary_keys</code>	Override the default primary keys for a table. Provide a list of column names.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 1, 2026**

Create a custom connector

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

This page shows how to create a connector for a source that isn't supported in Lakeflow Connect yet. First, build and test your connector locally using the tools and templates in the Lakeflow Community Connectors repository on GitHub. The repository includes AI-powered development tools to assist with each phase, including source research, authentication setup, implementation, and testing.

When your custom connector is ready to use, try it in your Databricks workspace, then register it with the community by opening a pull request.

To use a registered community connector, see [Use a registered community connector](#).

Requirements

Before you start, make sure you have:

- Python 3.10 or above
- A Databricks workspace with Unity Catalog enabled
- API credentials for the source you want to connect to
- Git installed locally

Set up the repository

Clone the [Lakeflow Community Connectors](https://github.com/databrickslabs/lakeflow-community-connectors) repository and install the development dependencies.

1. Clone the repository:

Bash

```
git clone https://github.com/databrickslabs/lakeflow-community-connectors.git
cd lakeflow-community-connectors
```

2. Create a virtual environment and install dependencies:

Bash

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

3. Copy an existing connector directory (for example, `connectors/stripe/`) as a starting point for your new connector:

Bash

```
cp -r connectors/stripe connectors/<your-source>
```

Implement the LakeflowConnect interface

Each community connector implements the `LakeflowConnect` interface, which defines how your connector authenticates, discovers tables, returns schemas, and reads data.

Python

```
class LakeflowConnect:
    def __init__(self, options: dict[str, str]) -> None:
        """Initialize with connection parameters"""
```

 Ask Genie

```

def list_tables(self) -> list[str]:
    """Return names of all tables supported by this connector."""

    def get_table_schema(self, table_name: str, table_options: dict[str,
str]) -> StructType:
        """Return the Spark schema for a table."""

    def read_table_metadata(self, table_name: str, table_options:
dict[str, str]) -> dict:
        """Return metadata: primary_keys, cursor_field, ingestion_type
(snapshot|cdc|cdc_with_deletes|append)."""

    def read_table(self, table_name: str, start_offset: dict,
table_options: dict[str, str]) -> (Iterator[dict],
dict):
        """Yield records as JSON dicts and return the next offset
for incremental reads."""

    def read_table_deletes(self, table_name: str, start_offset: dict,
table_options: dict[str, str]) ->
(Iterator[dict], dict):
        """Optional: Only required if ingestion_type is
'cdc_with_deletes'."""

```

Method descriptions

Method	Description
<code>__init__</code>	Receives the connection parameters as a dictionary and initializes the API client for your source.
<code>list_tables</code>	Returns the names of all tables (or API endpoints) your connector exposes. Databricks uses this list to populate the table selection UI.
<code>get_table_schema</code>	Returns a Spark <code>StructType</code> describing the schema for the given table. Called before the first pipeline run and on each run when schema evolution is enabled.
<code>read_table_metadata</code>	Returns a dictionary with <code>primary_keys</code> , <code>cursor_field</code> , and <code>ingestion_type</code> . The <code>ingestion_type</code> must be one  Ask Genie

Method	Description
	of <code>snapshot</code> , <code>cdc</code> , <code>cdc_with_deletes</code> , or <code>append</code> .
<code>read_table</code>	Yields records as Python dictionaries and returns the next offset for incremental reads. On the first run, <code>start_offset</code> is empty. On subsequent runs, it contains the offset returned by the previous run.
<code>read_table_deletes</code>	Optional. Only implement this method if <code>ingestion_type</code> is <code>cdc_with_deletes</code> . Yields deleted record keys and returns the next offset.

Develop your connector

Follow these steps to build and validate a new connector:

1. **Research the source API:** Study the source's API specifications, authentication mechanisms, rate limits, and available data schemas. Identify which tables or endpoints to expose.
2. **Set up authentication:** Generate the connection specification, configure credentials for the source, and verify connectivity from your development environment.
3. **Implement the connector:** Code all required `LakeflowConnect` interface methods to connect to the source API and return data in the expected format.
4. **Test and iterate:** Run the standard test suites against a real source system and fix any issues. See [Test your connector](#) for details.
5. **Document the connector:** Write a user-facing `README.md` and generate the connector spec YAML file that describes the connector's configurable parameters.
6. **Build the deployment artifact:** Run the build script to produce the single-file artifact that can be deployed in a workspace.

Test your connector

The repository provides several testing approaches:

Generic test suite (required)

Connects to a real source using your provided credentials to verify end-to-end functionality, including authentication, schema discovery, and data reads.

Bash

```
python -m pytest tests/generic/ --connector <your-source> --credentials  
credentials.json
```

Write-back testing (recommended)

Executes write-read-verify cycles to validate incremental reads and deletes. This confirms that your offset tracking and CDC logic work correctly.

Bash

```
python -m pytest tests/writeback/ --connector <your-source> --credentials  
credentials.json
```

Unit tests

Write unit tests for any complex custom logic in your connector, such as pagination handling, type coercion, or error recovery.

Build the deployment artifact

After your connector passes the test suites, run the merge script to generate a single-file deployment artifact. The pipeline uses this file at runtime rather than the full repository.

Bash

```
python tools/scripts/merge_python_source.py --connector <your-source>
```

This produces a self-contained Python file in `dist/<your-source>/` that includes all connector code and dependencies.

Create an ingestion pipeline

To try your connector:

1. In the sidebar of your Databricks workspace, click **+New > Add or upload data**, then select **+ Add Community Connector** under **Community connectors**.
2. For **Source name**, enter the name of your connector.
3. For **GitHub repository URL**, enter the URL of the GitHub repository that hosts your connector source code.
4. Click **Add Connector**.
5. Click **+ Create connection** or select an existing connection, then click **Next**.
6. For **Pipeline name**, enter a name for the pipeline.
7. For **Event log location**, enter a catalog name and a schema name. Databricks stores the pipeline event log here. Ingested tables are also written here by default.
8. For **Root path**, enter your workspace path (for example, `/Workspace/Users/<your-email>/connectors`). Databricks clones and stores the connector source code here.
9. Click **Create pipeline**.
10. In the pipeline editor, open `ingest.py` and modify the **objects** field to include the tables you want to ingest. For example:

Python

 Ask Genie

```

from databricks.labs.community_connector.pipeline import ingest

pipeline_spec = {
    "connection_name": "my_connector_connection", # Required: UC
    connection name
    "objects": [
        {"table": {"source_table": "my_table"}},
    ],
}

ingest(spark, pipeline_spec)

```

11. Run the pipeline manually or schedule it.

Pipeline configuration options

You can configure the following options in `ingest.py`:

Option	Description
<code>connection_name</code>	Required. The name of the connection that stores authentication credentials for the source.
<code>objects</code>	Required. A list of tables to ingest. Each entry has the format <code>{"table": {"source_table": "..."}}</code> . You can also specify an optional <code>destination_table</code> inside the <code>table</code> object.
<code>destination_catalog</code>	The catalog where ingested tables are written. Defaults to the catalog set during pipeline creation.
<code>destination_schema</code>	The schema where ingested tables are written. Defaults to the schema set during pipeline creation.
<code>scd_type</code>	The slowly changing dimension strategy: <code>SCD_TYPE_1</code> , <code>SCD_TYPE_2</code> , or <code>APPEND_ONLY</code> . Defaults to <code>SCD_TYPE_1</code> .

Option	Description
<code>primary_keys</code>	Override the default primary keys for a table. Provide a list of column names.

Register your connector

After you build and test your connector, open a pull request in the [Lakeflow Community Connectors](#) repository to make it available to the community.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

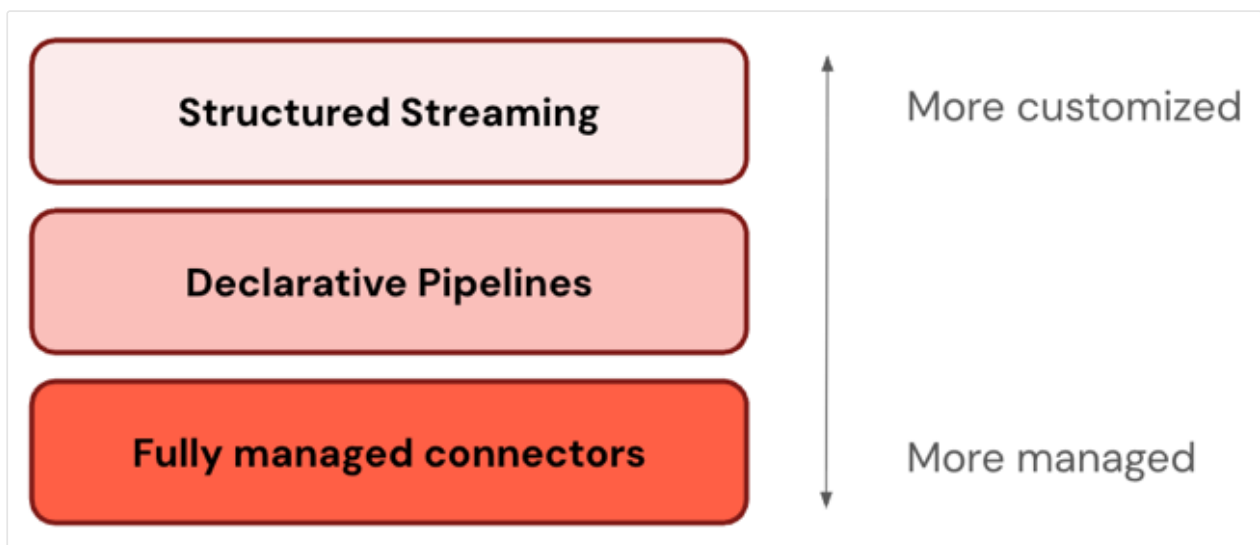
Last updated on **Mar 11, 2026**

Standard connectors in Lakeflow Connect

This page describes the standard connectors in Databricks Lakeflow Connect, which offer higher levels of ingestion pipeline customization compared to the managed connectors.

Layers of the ETL stack

Some connectors operate at one level of the ETL stack. For example, Databricks offers fully-managed connectors for enterprise applications like Salesforce and databases like SQL Server. Other connectors operate at multiple layers of the ETL stack. For example, you can use standard connectors in either Structured Streaming for full customization or Lakeflow Spark Declarative Pipelines for a more managed experience.



Databricks recommends starting with the most managed layer. If it doesn't satisfy your requirements (for example, if it doesn't support your data source), drop down to the next layer.

The following table describes the three layers of ingestion products, ordered from most customizable to most managed:

Layer	Description
Structured Streaming	Apache Spark Structured Streaming is a streaming engine that offers end-to-end fault tolerance with exactly-once processing guarantees using Spark APIs.
Lakeflow Spark Declarative Pipelines	Lakeflow Spark Declarative Pipelines builds on Structured Streaming, offering a declarative framework for creating data pipelines. You can define the transformations to perform on your data, and Lakeflow Spark Declarative Pipelines manages orchestration, monitoring, data quality, errors, and more. Therefore, it offers more automation and less overhead than Structured Streaming.
Managed connectors	Fully-managed connectors build on Lakeflow Spark Declarative Pipelines, offering even more automation for the most popular data sources. They extend Lakeflow Spark Declarative Pipelines functionality to also include source-specific authentication, CDC, edge case handling, long-term API maintenance, automated retries, automated schema evolution, and so on. Therefore, they offer even more automation for any supported data sources.

Choose a connector

The following table lists standard ingestion connectors by data source and level of pipeline customization. For a fully automated ingestion experience, use [managed connectors](#) instead.

SQL examples for incremental ingestion from cloud object storage use `CREATE STREAMING TABLE` syntax. It offers SQL users a scalable and robust ingestion experience, therefore it's the recommended alternative to `COPY INTO`.

Source	More customization	Some customization	More automation
Cloud object storage	Auto Loader with Structured Streaming  Python  Scala	Auto Loader with Lakeflow Spark Declarative Pipelines  Python  SQL	Auto Loader with Databricks SQL  SQL
SFTP servers	Ingest files from SFTP servers (Python, SQL)	N/A	N/A
Apache Kafka	Structured Streaming with Kafka source  Python  Scala	Lakeflow Spark Declarative Pipelines with Kafka source  Python  SQL	Databricks SQL with Kafka source  SQL
Amazon Kinesis	Structured Streaming with Kinesis source  Python  Scala	Lakeflow Spark Declarative Pipelines with Kinesis source  Python  SQL	Databricks SQL with Kinesis source  SQL
Google Pub/Sub	Structured Streaming with Pub/Sub source  Python  Scala	Lakeflow Spark Declarative Pipelines with Pub/Sub source  Python  SQL	Databricks SQL with Pub/Sub source  SQL  Ask Genie

Source	More customization	Some customization	More automation
Apache Pulsar	Structured Streaming with Pulsar source ☒ Python ☒ Scala	Lakeflow Spark Declarative Pipelines with Pulsar source ☒ Python ☒ SQL	Databricks SQL with Pulsar source ☒ SQL

Ingestion schedules

You can configure ingestion pipelines to run on a recurring schedule or continuously.

Use case	Pipeline mode
Batch ingestion	Triggered : Processes new data on a schedule or when manually triggered.
Streaming ingestion	Continuous : Processes new data as it arrives in the source.

Is this page

as this page helpful?

☒ Yes

☐ No

☐ Send feedback

Last updated on **Jan 20, 2026**

Ingest data from cloud object storage

This article lists the ways you can configure incremental ingestion from cloud object storage.

Add data UI

To learn how to use the add data UI to create a managed table from data in cloud object storage, see [Load data using a Unity Catalog external location](#).

Notebook or SQL editor

This section describes options for configuring incremental ingestion from cloud object storage using a notebook or the Databricks SQL editor.

Auto Loader

[Auto Loader](#) incrementally and efficiently processes new data files as they arrive in cloud storage without additional setup. Auto Loader provides a Structured Streaming source called `cloudFiles`. Given an input directory path on the cloud file storage, the `cloudFiles` source automatically processes new files as they arrive, with the option of also processing existing files in that directory.

COPY INTO

With [COPY INTO](#), SQL users can idempotently and incrementally ingest data from cloud object storage into Delta tables. You can use `COPY INTO` in Databricks

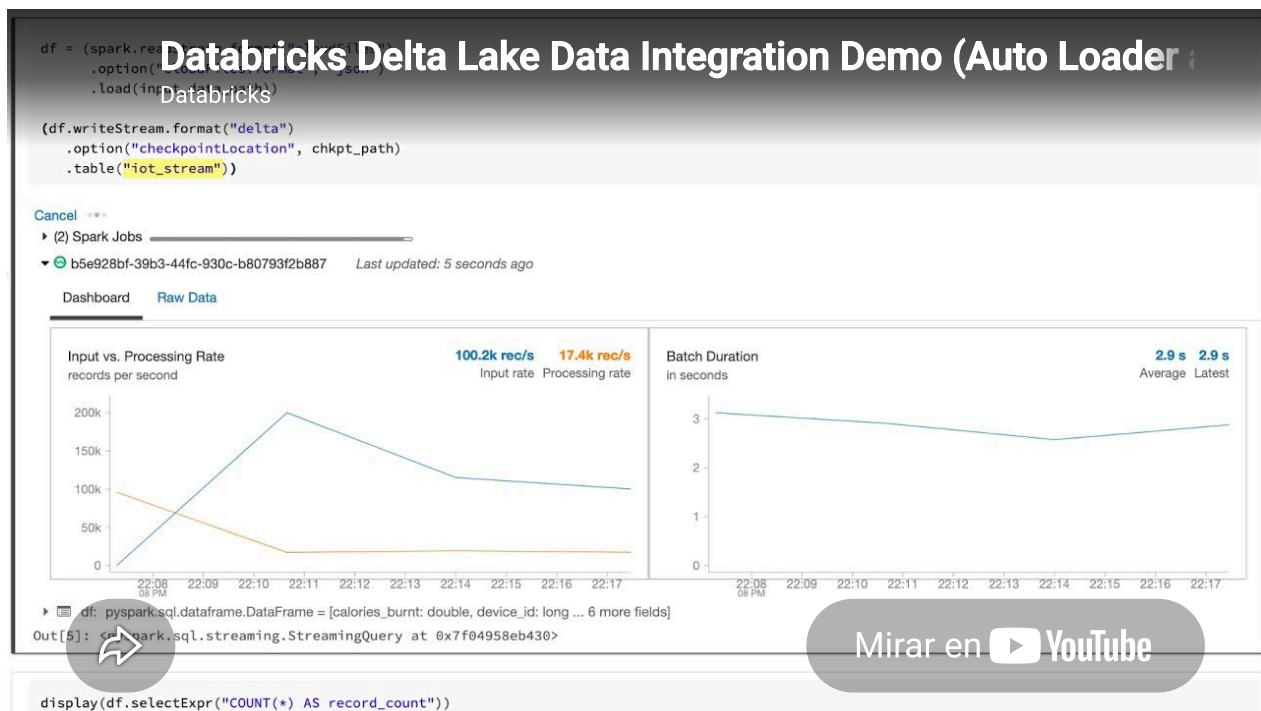
notebooks, and Lakeflow Jobs.

When to use COPY INTO and when to use Auto Loader

Here are a few things to consider when choosing between Auto Loader and `COPY INTO`:

- If you're going to ingest files in the order of thousands over time, you can use `COPY INTO`. If you are expecting files in the order of millions or more over time, use Auto Loader. Auto Loader requires fewer total operations to discover files compared to `COPY INTO` and can split the processing into multiple batches, which means that Auto Loader is less expensive and more efficient at scale.
- If your data schema is going to evolve frequently, Auto Loader provides better primitive data types around schema inference and evolution. See [Configure schema inference and evolution in Auto Loader](#) for more details.
- Loading a subset of re-uploaded files can be a bit easier to manage with `COPY INTO`. With Auto Loader, it's harder to reprocess a select subset of files. However, you can use `COPY INTO` to reload the subset of files while an Auto Loader stream is running simultaneously.
- For an even more scalable and robust file ingestion experience, Auto Loader enables SQL users to leverage streaming tables. See [Use standalone streaming tables](#).

For a brief overview and demonstration of Auto Loader and `COPY INTO`, watch the following YouTube video (2 minutes).



Automate ETL with Lakeflow Spark Declarative Pipelines and Auto Loader

You can simplify deployment of scalable, incremental ingestion infrastructure with Auto Loader and Lakeflow Spark Declarative Pipelines. Lakeflow Spark Declarative Pipelines does not use the standard interactive execution found in notebooks, instead it emphasizes deployment of infrastructure ready for production.

- [Tutorial: Build an ETL pipeline with Lakeflow Spark Declarative Pipelines](#)
- [Set up incremental ingestion from Amazon S3](#)
- [Streaming tables](#)

Third-party ingestion tools

Databricks validates technology partner integrations that enable you to ingest from various sources, including cloud object storage. These integrations enable low-code, scalable data ingestion from a variety of sources into Databricks. See [Technology partners](#). Some technology partners are featured in [What is Databricks Partner Connect?](#), which provides a UI that simplifies connecting third-party tools to your lakehouse data.

Is this page

as this page helpful?

☐ Yes	☐ No
☐ Send feedback	

Last updated on **May 11, 2026**

Ingest files from Google Drive

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

ⓘ COMPLIANCE

The Google Drive connector supports use in workspaces with the [Configure enhanced security and compliance settings](#) enabled.

The standard Google Drive connector in Lakeflow Connect allows you to use Databricks Spark and SQL functions (`read_files`, `spark.read`, `COPY INTO`, and Auto Loader) to create Spark dataframes, materialized views, and streaming tables directly from files in Google Drive.

With this approach, you can create custom pipelines for common file ingestion use cases:

- Streaming file ingestion (unstructured): Ingesting many source files (for example, PDFs, Google Docs, and Google Slides) into a single target table as binary data, ideal for RAG pipelines.
- Streaming file ingestion (structured): Merging many source files (for example, CSV and JSON files) into a single, structured target table.
- Batch file ingestion: Ingesting a single, specific file (such as a Google Sheet) or a batch of files into one target table.

These interfaces are supported:

- Declarative Automation Bundles
- Databricks APIs
- Databricks SDKs

- Databricks CLI

Limitations

- The connector is API-only and does not support pipeline creation in the Databricks UI.
- You can use the `pathGlobFilter` option to filter files by name (for example, `pathGlobFilter => '*.csv'`). Built-in Google formats (for example, Google Docs or Google Sheets) can't be filtered using this option. Folder path filtering is also not supported.
- Unsupported formats include Google Forms, Google Sites, Google Jams, and Google Vids. The ingestion process skips unsupported formats.
- Setting `recursiveFileLookup=false` is not supported for Spark batch reads. Using `recursiveFileLookup=false` will behave the same as `recursiveFileLookup=true`.

Requirements

Before you begin, verify that you have:

- A Unity Catalog-enabled workspace.
- `CREATE CONNECTION` privileges to create a Google Drive connection, or the appropriate privilege to use an existing one based on your [cluster access mode](#):
 - Dedicated Access Mode: `MANAGE CONNECTION`.
 - Standard Access Mode: `USE CONNECTION`.
- Databricks Runtime 17.3 or above.
- The Excel file format Beta feature enabled, if you want to ingest Google Sheets or Excel files. See [Read Excel files](#).
- A Google account with the necessary permissions to create a Google Cloud project.

Configure OAuth 2.0

Set up a Google Cloud project and enable the Google Drive API

1. Go to the [Google Cloud console](#).
2. Create a new project. You might be prompted to set up two-factor authentication.
3. Go to **APIs & Services > Library**.
4. Search for "Google Drive API".
5. Select **Google Drive API**.
6. Select **Enable**.

Set up the OAuth consent screen for your project

1. On the Google Cloud console home screen, go to **APIs & Services > OAuth Consent Screen**. You will see a message that says "Google Auth Platform not configured yet".
2. Select **Get Started**.
3. Fill in the **App Information** section. Enter any name for the app name (for example, `Databricks connection`). The support email can be any email in your organization.
4. Select **Next**.
5. In the **Audience** section, select **External**, then select **Next**.
6. Fill in the **Contact Information** section, then select **Next**.
7. Review the Google API Services User Data Policy, then select **Create**.
8. Go back to **Google Auth Platform > Data Access**.
9. Select **Add or remove scopes**.
10. Add the following scope in the **Manually add scopes** section, select **Add to table**, then select **Update**: `https://www.googleapis.com/auth/drive.readonly`
11. Select **Save**.

Create OAuth 2.0 client credentials

1. On the Google Cloud console home screen, go to **APIs & Services > Credentials**.
2. Select **Create Credentials > OAuth Client ID**.

3. Choose **Web Application** and set a custom name.
4. In **Authorized Redirect URIs**, select **Add URI**.
5. Add a redirect URI to `<databricks-instance-url>/login/oauth/google.html`, replacing `<databricks-instance-url>` with your Databricks instance URL. For example: `https://instance-name.databricks.com/login/oauth/google.html`
6. Select **Create**. A dialog with your credentials appears.
7. Record the following values. Alternatively, you can download the OAuth Client JSON file, which contains this information:
 - Client ID (format: `0123*****-*****.apps.googleusercontent.com`)
 - Client secret (format: `ABCD*-*****`)

Add test users to your project

1. Go to **Google Auth Platform > Audience**.
2. Under **Test users**, select **Add users**.
3. Add the email address of the Google account you will use to create the connection.

Create a connection

1. In the Databricks workspace, select **Catalog > External locations > Connections > Create connection**.
2. On the **Connection basics** page of the **Set up connection** wizard, specify a unique **Connection name**.
3. In the **Connection type** dropdown menu, search for and select **Google Drive**.
4. (Optional) Add a comment.
5. Select **Next**.
6. On the **Authentication** page, enter the following:
 - **OAuth scope:** `https://www.googleapis.com/auth/drive.readonly`  Ask Genie

- **Client secret:** The client secret from [Create OAuth 2.0 client credentials](#).
 - **Client ID:** The client ID from [Create OAuth 2.0 client credentials](#).
7. Select **Sign in with Google** and sign in with the Google account from [Add test users to your project](#).
 8. Select **Continue**, then select **Continue** again.
 9. After you're redirected back to the Databricks workspace, select **Create connection**.

Ingest files from Google Drive

Ingest files from Google Drive using the connector and `read_files` (Databricks SQL), Auto Loader (`.readStream` with `cloudFiles`), `COPY INTO`, and `spark.read`. You must provide the following values:

- The Google Drive URL as the path.
- The Unity Catalog connection using the `databricks.connection` data source option.
- The URL of the Google Drive resource you want to access. The URL can refer to a specific file, a folder, or an entire drive. For example:
 - `https://docs.google.com/spreadsheets/d/12345/edit?random_query_params_here`
 - `https://drive.google.com/drive/u/0/folders/12345`
 - `https://docs.google.com/document/d/12345/edit`
 - `https://drive.google.com/file/d/1kiXnHmU4Y8X66ijULky5EPDNCgtT14Ps/view?usp=drive_link`
 - `https://drive.google.com/drive/`
 - `https://drive.google.com/drive/my-drive`
 - `https://drive.google.com/`

Stream Google Drive files using Auto Loader

Auto Loader is the most efficient way to incrementally ingest structured files from Google Drive. It automatically detects new files and processes them as they arrive. It

can also ingest structured and semi-structured files such as CSV and JSON with automatic schema inference and evolution. For details about Auto Loader usage, see [Common data loading patterns](#).

Python

```
# Incrementally ingest new PDF files
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "binaryFile")
      .option("databricks.connection", "my_gdrive_conn")
      .option("cloudFiles.schemaLocation", <path to a schema location>)
      .option("pathGlobFilter", "*.pdf")
      .load("https://drive.google.com/drive/folders/1a2b3c4d...")
      .select("*", "_metadata")
)

# Incrementally ingest CSV files with automatic schema inference and evolution
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .option("databricks.connection", "my_gdrive_conn")
      .option("pathGlobFilter", "*.csv")
      .option("inferColumnTypes", True)
      .option("header", True)
      .load("https://drive.google.com/drive/folders/1a2b3c4d...")
)
```

Read Google Drive files using Spark batch read

The following example shows how to ingest Google Drive files in Python using the `spark.read` function. For a list of supported file formats and Spark reader options, see [Generic File Source Options](#) in the Apache Spark documentation.

Setting `recursiveFileLookup=false` is not supported for Spark batch reads. Using `recursiveFileLookup=false` will behave the same as `recursiveFileLookup=true`.

Python

```
# Read unstructured data as binary files
df = (spark.read
      .format("binaryFile")
      .option("databricks.connection", "my_gdrive_conn")
      .option("recursiveFileLookup", True))
```

 Ask Genie

```

.option("pathGlobFilter", "*.pdf")
.load("https://drive.google.com/drive/folders/1a2b3c4d..."))

# Read a batch of CSV files, infer the schema, and load the data into a
DataFrame
df = (spark.read
      .format("csv")
      .option("databricks.connection", "my_gdrive_conn")
      .option("pathGlobFilter", "*.csv")
      .option("recursiveFileLookup", True)
      .option("inferSchema", True)
      .option("header", True)
      .load("https://drive.google.com/drive/folders/1a2b3c4d..."))

```

Read a single structured Google Drive file

The following example reads a single tab of a Google Sheet and loads it into a DataFrame. It demonstrates a few common parsing options.

For a full list of supported parsing options for Excel files and Google Sheets, see [Read Excel files](#). For a full list of all other supported file formats and Spark reader options, see [Generic File Source Options](#) in the Apache Spark documentation.

Python

```

df = (spark.read
      .format("excel") # use 'excel' for Google Sheets
      .option("databricks.connection", "my_gdrive_conn")
      .option("headerRows", 1) # optional
      .option("inferColumns", True) # optional
      .option("dataAddress", "Sheet1!A1:Z10") # optional
      .load("https://docs.google.com/spreadsheets/d/9k8j7i6f..."))

```

Read Google Drive files using Spark SQL

The following example shows how to ingest Google Drive files in SQL using the `read_files` table-valued function. For details about `read_files` usage, see [read_files table-valued function](#).

SQL

 Ask Genie

```

-- Read pdf files
CREATE TABLE my_table AS
SELECT * FROM read_files(
  "https://drive.google.com/drive/folders/1a2b3c4d...",
  `databricks.connection` => "my_gdrive_conn",
  format => "binaryFile",
  pathGlobFilter => "*.pdf", -- optional. Example: only ingest PDFs
  schemaEvolutionMode => "none"
);

-- Read a Google Sheet and range
CREATE TABLE my_sheet_table AS
SELECT * FROM read_files(
  "https://docs.google.com/spreadsheets/d/9k8j7i6f...",
  `databricks.connection` => "my_gdrive_conn",
  format => "excel",
  headerRows => 1, -- optional
  dataAddress => "Sheet1!A2:D10", -- optional
  schemaEvolutionMode => "none"
);

```

Ingest Google Drive files using Lakeflow Spark Declarative Pipelines

NOTE

The Google Drive connector requires Databricks Runtime 17.3 or later. To use the connector, set `"CHANNEL" = "PREVIEW"` in your pipeline settings. For more information on previews, see [Pipeline properties reference](#).

This examples shows how to read Google Drive files using Auto Loader in Lakeflow Spark Declarative Pipelines. For more information, see [Lakeflow Spark Declarative Pipelines](#)

SQL Python

SQL

```

-- Incrementally ingest new PDF files
CREATE OR REFRESH STREAMING TABLE gdrive_pdf_table

```

 Ask Genie

```

AS SELECT * FROM STREAM read_files(
  "https://drive.google.com/drive/folders/1a2b3c4d...",
  format => "binaryFile",
  `databricks.connection` => "my_gdrive_conn",
  pathGlobFilter => "*.pdf");

-- Incrementally ingest CSV files with automatic schema inference and
-- evolution
CREATE OR REFRESH STREAMING TABLE gdrive_csv_table
AS SELECT * FROM STREAM read_files(
  "https://drive.google.com/drive/folders/1a2b3c4d...",
  format => "csv",
  `databricks.connection` => "my_gdrive_conn",
  pathGlobFilter => "*.csv",
  "header", "true");

-- Read a specific Excel file from Google Drive in a materialized view
CREATE OR REFRESH MATERIALIZED VIEW gdrive_excel_table
AS SELECT * FROM read_files(
  "https://docs.google.com/spreadsheets/d/9k8j7i6f...",
  `databricks.connection` => "my_gdrive_conn",
  format => "excel",
  headerRows => 1, -- optional
  dataAddress => "Sheet1!A2:D10", -- optional
  `cloudFiles.schemaEvolutionMode` => "none"
);

```

Parse unstructured files with

`ai_parse_document`

When you ingest unstructured files from Google Drive (such as PDFs, Word documents, or PowerPoint files) using the standard Google Drive connector with `binaryFile` format, the file contents are stored as raw binary data. To prepare these files for AI workloads—such as RAG, search, classification, or document understanding—you can parse the binary content into structured, queryable output using `ai_parse_document`.

The following example shows how to parse unstructured documents stored in a bronze Delta table named `documents`, adding a new column with parsed content:

SQL

```

CREATE TABLE documents AS
SELECT *, _metadata FROM read_files(

```

 Ask Genie

```

    "https://drive.google.com/drive/folders/1a2b3c4d...",
    `databricks.connection` => "my_gdrive_conn",
    format => "binaryFile",
    pathGlobFilter => "*.pdf,jpeg}"
  );
SELECT *, ai_parse_document(content) AS parsed_content
FROM documents;

```

The `parsed_content` column contains extracted text, tables, layout information, and metadata that can be directly used for downstream AI pipelines.

Incremental parsing with Lakeflow Spark Declarative Pipelines

You can also use `ai_parse_document` within Lakeflow Spark Declarative Pipelines to enable incremental parsing. As new files stream in from Google Drive, they are automatically parsed as your pipeline updates.

For example, you can define a materialized view that continuously parses newly ingested documents:

SQL

```

CREATE OR REFRESH STREAMING TABLE documents
AS SELECT *, "_metadata" FROM STREAM read_files(
  "https://drive.google.com/drive/folders/1a2b3c4d...",
  format => "binaryFile",
  `databricks.connection` => "my_gdrive_conn",
  pathGlobFilter => "*.pdf,jpeg");

CREATE OR REFRESH MATERIALIZED VIEW documents_parsed
AS
SELECT *, ai_parse_document(content) AS parsed_content
FROM documents;

```

This approach ensures that:

- Newly ingested Google Drive files are parsed automatically whenever the materialized view refreshes
- Parsed outputs stay in sync with incoming data

- Downstream AI pipelines always operate on up-to-date document representations

For supported formats and advanced options, see `ai_parse_document` function.

How built-in Google formats are handled

You do not need to export your built-in Google files (Docs, Sheets) manually. The connector automatically exports them to an open format during ingestion.

Google format	Exported as (default)
Google Docs	<code>application/vnd.openxmlformats-officedocument.wordprocessingml.document</code> (DOCX)
Google Sheets	<code>application/vnd.openxmlformats-officedocument.spreadsheetml.sheet</code> (XLSX)
Google Slides	<code>application/vnd.openxmlformats-officedocument.presentationml.presentation</code> (PPTX)
Google Drawings	<code>application/pdf</code> (PDF)

Google Drive export format configuration

You can configure how Google Drive native files are exported by setting Spark configurations using `spark.conf.set()`. These configurations determine the `MIME` type used when exporting Google Docs, Sheets, Slides, and Drawings.

Configuration keys:

- `fs.gdrive.format.document.export`: Google Docs export format.
- `fs.gdrive.format.spreadsheet.export`: Google Sheets export format.

 Ask Genie

- `fs.gdrive.format.presentation.export`: Google Slides export format.
- `fs.gdrive.format.drawing.export`: Google Drawings export format.

For a complete list of supported export formats, see [Export MIME types for Google Workspace documents](#) in the Google Workspace documentation.

The following example exports a Google Docs file as TXT.

Python

```
spark.conf.set("fs.gdrive.format.document.export", "text/plain")
df = spark.read.text("https://docs.google.com/document/d/1a2b3c4d...")
```

The following example exports a Google Sheets file as CSV.

Python

```
spark.conf.set("fs.gdrive.format.spreadsheet.export", "text/csv")
df = spark.read.option("header",
"true").csv("https://docs.google.com/spreadsheets/d/1a2b3c4d...")
```

Schemas

binaryFile format

When you use `format => 'binaryFile'`, the resulting table has the following schema:

- `path` (string): The full URL to the file.
- `modificationTime` (timestamp): The last modified time of the file.
- `length` (long): The size of the file in bytes.
- `content` (binary): The raw binary content of the file.

`_metadata` column

You can get metadata information for input files with the `_metadata` column such as `file_name`, `file_path`, `file_size`, and `file_modification_time`. The `_metadata` column is a hidden column, and is available for all input file formats. To include the

`_metadata` column in the returned DataFrame, you must explicitly select it in the read query where you specify the source. For more information, see [File metadata column](#).

For Google Drive native files (such as Google Docs, Google Sheets, and Google Slides), the `file_size` field refers to the size of the file stored on Google Drive, not the size of the file in the exported format (for example, DOCX, XLSX, or PPTX).

Example of selecting `_metadata`:

SQL

```
SELECT *, _metadata FROM read_files(  
  "https://drive.google.com/",  
  `databricks.connection` => "my_connection",  
  format => "binaryFile"  
);
```

Google Drive metadata column

ⓘ PREVIEW

This feature is in [Private Preview](#). To try it, reach out to your Databricks contact.

The `_gdrive_metadata` column is a hidden metadata column that provides access to Google Drive-specific properties of ingested files, sourced from the [Google Drive files resource](#). It requires Databricks Runtime 18.1 or above and is available for all file formats when reading from Google Drive. To include the `_gdrive_metadata` column in the returned DataFrame, you must explicitly select it in the read query.

If the data source contains a column named `_gdrive_metadata`, the Google Drive metadata column is renamed to `__gdrive_metadata` (with an extra leading underscore) to deduplicate. Additional underscores are added until the name is unique.

Common file metadata like the file path or size can be queried using the `_metadata` column. For more information, see [File metadata column](#).

⚠ WARNING

 Ask Genie

New fields might be added to the `_gdrive_metadata` column in future releases. To prevent schema evolution errors if the `_gdrive_metadata` column is updated, you can select specific fields from the column in your queries. See [Examples](#).

Schema

The `_gdrive_metadata` column is a `STRUCT` containing the following fields. All fields are nullable.

Name	Type	Description	Example
<code>id</code>	<code>STRING</code>	The Google Drive file ID.	<code>1pCzw0Apmv</code>
<code>drive_id</code>	<code>STRING</code>	The ID of the shared drive that contains the file. <code>null</code> for files in a user's My Drive.	<code>0ABpL6n51H</code>
<code>parent_id</code>	<code>STRING</code>	The ID of the file's parent folder. Google Drive files can have multiple parents; this returns the first.	<code>1a2b3c4d5e</code>
<code>web_url</code>	<code>STRING</code>	The browser URL of the file on Google Drive.	<code>https://dr i4E7mYf2fe</code>

Name	Type	Description	Example
mime_type	STRING	The MIME type of the file. For Google Workspace files, this is the native Google type (for example, <code>application/vnd.google-apps.document</code>), not the exported type.	<code>text/csv</code>
md5_checksum	STRING	The MD5 checksum of the file's content. Only populated for binary files; <code>null</code> for Google Workspace files.	<code>06ffb3e392</code>
version	STRING	A monotonically increasing version number for the file.	<code>12</code>
created_timestamp	TIMESTAMP	The time the file was created.	<code>2025-12-01</code>
last_modified_by_email	STRING	The email of the user who last modified the file.	<code>alice@example</code>
last_modified_by_name	STRING	The display name of the user who last modified the file.	<code>Alice Exam</code>
shared	BOOLEAN	Whether the file has been shared with users other than the owner.	<code>true</code>
 Ask Genie			

Name	Type	Description	Example
properties	VARIANT	Custom public properties set on the file. See Properties in the Drive API .	<code>{"departme</code>
additional_metadata	VARIANT	Any other file resource fields returned by the Drive API but not extracted above.	<code>{"capabili</code>

NOTE

The `properties` and `additional_metadata` fields are returned as `VARIANT`. See [VARIANT](#) type.

Examples

The following examples show how to include the `_gdrive_metadata` column in a read query, select specific fields from the column, and extract values from `VARIANT` fields.

Python **SQL**

Python

```
df = (spark.read
      .format("binaryFile")
      .option("databricks.connection", "my_gdrive_conn")
      .load("https://drive.google.com/drive/folders/1a2b3c4d...")
      .select("*", "_metadata", "_gdrive_metadata"))
```

Select specific fields from the `_gdrive_metadata` struct:

 Ask Genie

Python

```
df = (spark.read
      .format("binaryFile")
      .option("databricks.connection", "my_gdrive_conn")
      .load("https://drive.google.com/drive/folders/1a2b3c4d...")
      .select("_gdrive_metadata.id", "_gdrive_metadata.md5_checksum"))
```

Extract values from the `properties` or `additional_metadata` `VARIANT` fields using the `::` cast operator:

SQL

```
SELECT
  *,
  _gdrive_metadata.properties:department::STRING AS department
FROM read_files(
  "https://drive.google.com/drive/folders/1a2b3c4d...",
  `databricks.connection` => "my_gdrive_conn",
  format => "binaryFile"
);
```

FAQ

I have a folder of multiple structured files (for example, multiple Google Sheets). How do I load each sheet or file as its own Delta table?

You must create a new ingestion query for each file to ingest into its own Delta table.

My files require custom parsing. How do I provide these parsing parameters to ensure my files are read correctly?

The connector supports all file format options available in Auto Loader, `COPY INTO`, and Spark. For details, see the following:

- [Auto Loader](#)
- [Basic Options](#)
- [Data sources – Spark](#)

Are files in subfolders recursively ingested?

When you use Auto Loader APIs (`spark.readStream` and `read_files`), all subfolders are recursively discovered and ingested. This is also true for batch `spark.read`, where files in the provided folder path are always read recursively.

My Google Sheet has many irregularities and requires specific parsing and cell-range extraction (for example, multiple tables per sheet). The schema cannot be inferred automatically. How do I handle this?

You can use Excel file format parsing options to parse your Google sheet file to your desired format. See [Read Excel files](#).

Alternatively, you can disable schema inference on Auto Loader, PySpark, or `read_files`. The resulting table has default column names, all data types are cast to a string, and the table might be sparse. You can then perform any necessary transformations downstream.

Next steps

- Explore [Common data loading patterns](#)
- Review `ai_parse_document` function

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 8, 2026**

Ingest files from SFTP servers

⚠️ PREVIEW

The SFTP connector is in [Public Preview](#).

Learn how to ingest files from SFTP servers using Lakeflow Connect. The SFTP connector extends Auto Loader functionality to provide secure, incremental ingestion from SFTP servers with Unity Catalog governance.

Key features

The SFTP connector offers the following:

- Private key and password-based authentication.
- Incremental file ingestion and processing with exactly-once guarantees.
- Automatic schema inference, evolution, and data rescue.
- Unity Catalog governance for secure ingestion and credentials.
- Wide file format support: `JSON`, `CSV`, `XML`, `PARQUET`, `AVRO`, `TEXT`, `BINARYFILE`, and `ORC`.
- Built-in support for pattern and wildcard matching to easily target data subsets.
- Availability on all compute types, including Lakeflow Spark Declarative Pipelines, Databricks SQL, serverless and classic with Databricks Runtime 17.3 and above.

Before you begin

To create the connection and the ingestion pipeline, you must have the following:

- A workspace with Unity Catalog enabled.

- `CREATE CONNECTION` privileges to create an SFTP connection, or the appropriate privilege to use an existing one based on your [cluster access mode](#):
 - Dedicated Access Mode: `MANAGE CONNECTION`.
 - Standard Access Mode: `USE CONNECTION`.
- Compute that uses Databricks Runtime version 17.3 or above.

Configure SFTP

First, confirm that the source SFTP server is accessible to your Databricks cluster environment:

- Verify that the remote server is available in the VPC that has been configured with your workspace.
- Verify that your SSH rules allow for the IP range of the Databricks VPC (if you're using classic compute) or the stable IPs (if you're using serverless compute).
- From the classic compute plane, set up a stable IP address with a load balancer, NAT gateway, internet gateway, or equivalent, and connect it to the subnet where Databricks compute is deployed. This allows the compute resource to share a stable public IP address that can be allowlisted on the SFTP server side. For instructions to configure network settings, see [VPC peering](#).
- From serverless compute plane, see [Step 1: Create a network connectivity configuration and copy the stable IPs](#) to obtain stable egress IP.

Create the connection

Create a Unity Catalog connection to store your SFTP credentials. You must have `CREATE CONNECTION` privileges.

The connector supports the following authentication methods:

- PEM private key
- Password-based authentication

Databricks recommends using PEM private key authentication. Databricks also recommends using credentials with the least privilege on the source SFTP server (for example, a non-root user limited to read-only access).

When you create the pipeline, the connector attempts to automatically find a connection that you can use and that matches the host. If there are multiple matching connections, the connector chooses the first connection that successfully connects to the host. Starting with DBR 18.2, you can explicitly specify the connection using the `databricks.connection` option. This is the recommended option and avoids ambiguity when multiple connections exist for the same host.

PEM private key (recommended)

Catalog Explorer **SQL**

1. In the Databricks workspace, click  **Catalog**.
2. Click  **Connect**, then click **Connections**.
3. Click the **Create connection** button.
4. On the **Connection basics** page of the **Set up connection** wizard, enter a unique **Connection name**.
5. For **Connection type** select **SFTP**.
6. For **Auth type**, select **PEM Private Key**.
7. Select **Next**.
8. On the **Authentication** page, for **Host**, enter the hostname of the foreign server.
9. For **User**, enter the user identity used to access the foreign instance.
10. Select **Next**.
11. On the **Connection details** page, enter the private key in PEM format. Also provide the key passphrase, if applicable.
12. If you want to skip host key fingerprint check, clear **Enforce host key fingerprint**.

When this is selected, the connection only proceeds if the server's public key matches the expected SHA-256 fingerprint. When disabled, the connection

proceeds regardless of the match. Check with your network administrator before you disable this.

13. If **Enforce host key fingerprint** is checked, enter the SFTP server fingerprint.

You can retrieve the fingerprint from your server administrator or by using CLI commands. You can also press **Test and create connection** > **Test**. The resulting error message provides the fingerprint. For example:

```
ECDSA key fingerprint is SHA256:XXX/YYY
```

14. Select **Test and create connection**.
15. If the connection is successful, click **Create**.

Password-based authentication

Catalog Explorer **SQL**

1. In the Databricks workspace, click  **Catalog**.
2. Click  **Connect**, then click **Connections**.
3. Click the **Create connection** button.
4. On the **Connection basics** page of the **Set up connection** wizard, enter a unique **Connection name**.
5. For **Connection type** select **SFTP**.
6. For **Auth type**, select **Username and password**.
7. Select **Next**.
8. On the **Authentication** page, for **Host**, enter the hostname of the foreign server.
9. For **User**, enter the user identity used to access the foreign instance.
10. For **Password**, enter the password of the foreign instance.

11. Select **Next**.

12. If you want to skip host key fingerprint check, clear **Enforce host key fingerprint**.

When this is selected, the connection only proceeds if the server's public key matches the expected SHA-256 fingerprint. When disabled, the connection proceeds regardless of the match. Check with your network administrator before you disable this.

13. If **Enforce host key fingerprint** is checked, enter the SFTP server fingerprint.

You can retrieve the fingerprint from your server administrator or by using CLI commands. You can also press **Test and create connection** > **Test**. The resulting error message provides the fingerprint. For example:

```
ECDSA key fingerprint is SHA256:XXX/YYY
```

14. Select **Test and create connection**.

15. If the connection is successful, click **Create**.

Read files from the SFTP server

The following examples show how to read files from an SFTP server using Auto Loader's streaming capabilities. For details about Auto Loader usage, see [Common data loading patterns](#).

You can specify the Unity Catalog connection for authentication in one of two ways:

- **Explicit (recommended, Requires DBR 18.2 and above):** Specify the connection by name using the `databricks.connection` option. When you specify the connection, the host and port in the URI must match the stored connection credentials. The username field in the URI is optional. If omitted, the connector uses the username from the connection.
- **Automatic:** If you do not specify the connection by using the `databricks.connection` option, the connector resolves the connection by matching the `<host>` and `<username>` in the URI against available connections. If

there are multiple matching connections, the connector uses the first one that successfully connects.

The following example shows how to read files using automatic connection resolution:

Python

```
# Run the Auto Loader job to ingest all existing data in the SFTP server.
# The <username> and <host> in the URI must match the connection created
# in the previous step.
# The connector automatically resolves the matching Unity Catalog
# connection for authentication.
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.schemaLocation", "<path to store schema
information>") # This is a cloud storage path
      .option("cloudFiles.format", "csv") # Or other format supported by Auto
Loader
      # Specify the absolute path on the SFTP server starting from the root
      /.
      # Example: /home/<username>/data/files or /uploads/csv_files
      .load("sftp://<username>@<host>:<port>/<absolute_path_to_files>")
      .writeStream
      .format("delta")
      .option("checkpointLocation", "<path to store checkpoint information>")
# This is a cloud storage path.
      .trigger(availableNow = True)
      .table("<table name>"))
df.awaitTermination()
```

The following example shows how to explicitly specify the Unity Catalog connection using the `databricks.connection` option. This requires DBR 18.2 or above.

Python

```
# Requires DBR 18.2 or above. Explicitly specify the Unity Catalog
# connection by name (recommended).
# The username is optional in the URI when databricks.connection is
# provided.
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.schemaLocation", "<path to store schema
information>") # This is a cloud storage path
      .option("cloudFiles.format", "csv") # Or other format supported by Auto
Loader
      .option("databricks.connection", "<connection_name>")
      # Specify the absolute path on the SFTP server starting from the root
```

 Ask Genie

```

/.
# Example: /uploads/csv_files
.load("sftp://<host>:<port>/<absolute_path_to_files>")
.writeStream
.format("delta")
.option("checkpointLocation", "<path to store checkpoint information>")
# This is a cloud storage path.
.trigger(availableNow = True)
.table("<table name>"))
df.awaitTermination()

```

The following examples show how to read files from an SFTP server using Auto Loader in Lakeflow Spark Declarative Pipelines:

Python SQL

Python

```

from pyspark import pipelines as dp

# The <username> and <host> in the URI must match the connection created
# in the previous step.
# The connector automatically resolves the matching Unity Catalog
# connection for authentication.
@dp.table
def sftp_bronze_table():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv") # Or other format supported by
            Auto Loader
            # Specify the absolute path on the SFTP server starting from the root
            /.
            # Example: /home/username/data/files or /uploads/csv_files
            .load("sftp://<username>@<host>:<port>/<absolute_path_to_files>"))

```

Configure [Auto Loader options](#). All options are supported except:

- `cloudFiles.useNotifications`
- `cloudFiles.useManagedFileEvents`
- `cloudFiles.cleanSource`
- Cloud-specific options

Limitations

- SFTP is not supported across other ingestion surfaces, including `COPY INTO`, `spark.read`, and `dbutils.ls`.
- Writing back to an SFTP server is not supported.
- Auto Loader `cleanSource` (deleting or archiving files at source after ingestion) is not supported.
- The FTP protocol is not supported.

FAQ

Find answers to frequently asked questions about the SFTP connector.

How do I use wildcards or filename patterns to select files to ingest?

The SFTP connector builds on the standard Auto Loader framework to read from SFTP servers. This means that all Auto Loader options are supported. For file name patterns and wildcards, use the `pathGlobFilter` or `fileNamePattern` options. See [Auto Loader](#).

Can the SFTP connector ingest encrypted files? (Is PGP supported?)

The connector doesn't decrypt in flight, but you can ingest the encrypted files as binary files and decrypt them after ingestion.

How do I handle incompatible private key formats?

Only PEM format is supported. You can generate a private key in PEM format by doing one of the following:

- (Option 1) Create a new RSA key in the standard PEM format:

Shell

```
% ssh-keygen -t rsa -m pem
```

- (Option 2) Convert the existing OpenSSH-format key to PEM format:

Shell

```
% ssh-keygen -p -m pem -f /path/to/key # This updates the key file.
```

Additional resources

- [What is Auto Loader?](#)
- [Load data in pipelines](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 1, 2026**

Ingest files from SharePoint

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

ⓘ COMPLIANCE

The SharePoint connector supports use in workspaces with the [Configure enhanced security and compliance settings](#) enabled.

You can ingest structured, semi-structured, and unstructured files from Microsoft SharePoint into Delta tables. The SharePoint connector supports incremental ingestion of SharePoint files using batch and streaming APIs, including Auto Loader, `spark.read` and `COPY INTO`, all with Unity Catalog governance.

Choose your SharePoint connector

Lakeflow Connect offers two complementary SharePoint connectors. They both access data in SharePoint, but they support distinct goals.

Consideration	Managed SharePoint connector	Standard SharePoint connector
Management and customization	A fully-managed connector. Simple, low-maintenance connectors for enterprise applications that ingest data in to Delta tables and keep them in sync with the	Build custom ingestion pipelines with SQL, PySpark, or Lakeflow Spark Declarative Pipelines using batch and streaming APIs such as <code>read_files</code>, <code>spark</code>, <code>INTO</code>, and Auto Load  Ask Genie

Consideration	Managed SharePoint connector	Standard SharePoint connector
	source. See Managed connectors in Lakeflow Connect .	Offers the flexibility to perform complex transformations during ingestion, while giving you greater responsibility for managing and maintaining your pipelines.
Output format	Uniform binary content table. Ingests each file in binary format (one file per row), along with file metadata in additional columns.	Structured Delta tables. Ingests structured files (like CSV and Excel) as Delta tables. Can also be used to ingest unstructured files in binary format.
Granularity, filtering, and selection	No subfolder or file level selection today. No pattern-based filtering. Ingests all files in the specified SharePoint document library.	Granular and custom. URL-based selection to ingest from document libraries, subfolders, or individual files. Also supports pattern-based filtering using the <code>pathGlobFilter</code> option.

Key features

The standard SharePoint connector offers:

- Ingestion of structured, semi-structured, and unstructured files
- Granular ingestion: Ingest a specific site, a sub-site, a document library, a folder, or a single file
- Batch and streaming ingestion using `spark.read`, Auto Loader, and `COPY INTO`
- Automatic schema inference and evolution for structured and semi-structured formats such as CSV and Excel

- Secure credentials storage with a Unity Catalog connection
- File selection with pattern matching using `pathGlobFilter`

Requirements

To ingest files from SharePoint, you must have the following:

- A workspace with Unity Catalog enabled.
- `CREATE CONNECTION` privileges to create a SharePoint connection, or the appropriate privilege to use an existing one based on your [cluster access mode](#):
 - Dedicated Access Mode: `MANAGE CONNECTION`.
 - Standard Access Mode: `USE CONNECTION`.
- Compute that uses Databricks Runtime version 17.3 LTS or later.
- OAuth authentication set up with the `Sites.Read.All` or `Sites.Selected` permission scope.
- The SharePoint Beta feature is enabled from the **Previews** page. See [Manage Databricks previews](#).
- Optional: Enable the Excel Beta feature for parsing Excel files. See [Read Excel files](#).

Create the connection

Create a Unity Catalog connection to store your SharePoint credentials. The connection setup process is shared between both the standard and managed SharePoint connectors.

For complete connection setup instructions including OAuth authentication options, see [Overview of SharePoint ingestion setup](#).

Read files from SharePoint

To read files, pass the connection you created using the `databricks.connection` option and a URL that points to the SharePoint resource you want to access. The URL you provide determines the scope of the ingestion.

The following path types are supported on Databricks Runtime 17.3 LTS and later:

Path type	Description
Site	Copy the site URL from the address bar. <pre>https://mytenant.sharepoint.com/sites/test-site</pre>
Subsite	Copy the subsite URL from the address bar. <pre>https://mytenant.sharepoint.com/sites/test-site/test-subsite</pre>
Document library	Open the library from Site contents and copy the URL from the address bar. <pre>https://mytenant.sharepoint.com/sites/test-site/Shared%20Documents</pre> <pre>https://mytenant.sharepoint.com/sites/test-site/custom-drive</pre>
Folder	Open the folder from Site contents and copy the URL from the address bar. Alternatively, open the folder's Details pane in SharePoint and click the copy icon next to Path. <pre>https://mytenant.sharepoint.com/sites/test-site/Shared%20Documents/Forms/AllItems.aspx?id=%2Fsites...</pre> <pre>https://mytenant.sharepoint.com/sites/test-site/custom-drive/test-folder</pre>
File	Select the file, click the overflow menu (...), and select Preview. Copy the URL from the address bar. Alternatively, open the file's Details pane in SharePoint and click the copy icon next to Path. <pre>https://mytenant.sharepoint.com/sites/test-site/Shared%20Documents/Forms/AllItems.aspx?viewid=1a2b3c...</pre> <pre>https://mytenant.sharepoint.com/sites/test-site/custom-drive/test-folder/test.csv</pre>

Databricks Runtime 18.3 and later adds support for the following path types:

Path type	Description
Tenant	Copy the tenant root URL from the address bar. <code>https://mytenant.sharepoint.com</code>
Nested subsite	Copy the subsite URL from the address bar. <code>https://mytenant.sharepoint.com/sites/test-site/subsite/nested-subsite/nested-nested-subsite</code>
Share link	Select the file or folder, click the overflow menu (...), and select Copy link . Databricks recommends setting the share link to never expire. <code>https://mytenant.sharepoint.com/:i:/s/test-site/1A2B3C4D5E6F7G8H9I</code>
Microsoft 365 for the web (formerly Office)	Open the file in Microsoft 365 for the web and copy the URL from the address bar. <code>https://mytenant.sharepoint.com/:x:/r/sites/test-site/_layouts/15/Doc.aspx?sourcedoc=%1A2B...</code>

Examples

There are a few ways to read files using the standard SharePoint connector.

Stream SharePoint files using Auto Loader

Auto Loader provides the most efficient way to incrementally ingest structured files from SharePoint. It automatically detects new files and processes them as they arrive. It can also ingest structured and semi-structured files such as CSV and JSON with automatic schema inference and evolution. For details about Auto Loader usage, see [Common data loading patterns](#).

Python

```
# Incrementally ingest new PDF files
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "binaryFile")
      .option("databricks.connection", "my_sharepoint_conn")
      .option("cloudFiles.schemaLocation", <path to a schema location>)
      .option("pathGlobFilter", "*.pdf")

      .load("https://mytenant.sharepoint.com/sites/Marketing/Shared%20Documents")
)

# Incrementally ingest CSV files with automatic schema inference and
# evolution
df = (spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .option("databricks.connection", "my_sharepoint_conn")
      .option("pathGlobFilter", "*.csv")
      .option("inferColumnTypes", True)
      .option("header", True)

      .load("https://mytenant.sharepoint.com/sites/Engineering/Data/IoT_Logs")
)
```

Read SharePoint files using Spark batch read

The following example shows how to ingest SharePoint files in Python using the `spark.read` function.

Python

```
# Read unstructured data as binary files
df = (spark.read
      .format("binaryFile")
      .option("databricks.connection", "my_sharepoint_conn")
      .option("recursiveFileLookup", True)
      .option("pathGlobFilter", "*.pdf") # optional. Example: only ingest
      .load("https://mytenant.sharepoint.com/sites/Marketing/Shared%20Docu

# Read a batch of CSV files, infer the schema, and load the data into a Data
df = (spark.read
      .format("csv")
      .option("databricks.connection", "my_sharepoint_conn")
      .option("pathGlobFilter", "*.csv")
```

 Ask Genie

```

.option("recursiveFileLookup", True)
.option("inferSchema", True)
.option("header", True)
.load("https://mytenant.sharepoint.com/sites/Engineering/Data/IoT_Lc

# Read a specific Excel file from SharePoint, infer the schema, and load the
a DataFrame
df = (spark.read
    .format("excel")
    .option("databricks.connection", "my_sharepoint_conn")
    .option("headerRows", 1) # optional
    .option("dataAddress", "Sheet1!A1:M20") # optional

.load("https://mytenant.sharepoint.com/sites/Finance/Shared%20Documents/Mont
Oct.xlsx"))

```

Read SharePoint files using Spark SQL

The following example shows how to ingest SharePoint files in SQL using the `read_files` table-valued function. For details about `read_files` usage, see [read_files table-valued function](#).

SQL

```

-- Read pdf files
CREATE TABLE my_table AS
SELECT * FROM read_files(
    "https://mytenant.sharepoint.com/sites/Marketing/Shared%20Documents",
    `databricks.connection` => "my_sharepoint_conn",
    format => "binaryFile",
    pathGlobFilter => "*.pdf", -- optional. Example: only ingest PDFs
    schemaEvolutionMode => "none"
);

-- Read a specific Excel sheet and range
CREATE TABLE my_sheet_table AS
SELECT * FROM read_files(

"https://mytenant.sharepoint.com/sites/Finance/Shared%20Documents/Monthly/Re
Oct.xlsx",
    `databricks.connection` => "my_sharepoint_conn",
    format => "excel",
    headerRows => 1, -- optional
    dataAddress => "Sheet1!A2:D10", -- optional

```

```
schemaEvolutionMode => "none"
);
```

Incremental ingestion with COPY INTO

`COPY INTO` provides idempotent incremental loading of files into a Delta table. For details about `COPY INTO` usage, see [Common data loading patterns using COPY INTO](#).

SQL

```
CREATE TABLE IF NOT EXISTS sharepoint_pdf_table;
CREATE TABLE IF NOT EXISTS sharepoint_csv_table;
CREATE TABLE IF NOT EXISTS sharepoint_excel_table;

# Incrementally ingest new PDF files
COPY INTO sharepoint_pdf_table
  FROM "https://mytenant.sharepoint.com/sites/Marketing/Shared%20Documents"
  FILEFORMAT = BINARYFILE
  PATTERN = '*.pdf'
  FORMAT_OPTIONS ('databricks.connection' = 'my_sharepoint_conn')
  COPY_OPTIONS ('mergeSchema' = 'true');

# Incrementally ingest CSV files with automatic schema inference and evolution
COPY INTO sharepoint_csv_table
  FROM "https://mytenant.sharepoint.com/sites/Engineering/Data/IoT_Logs"
  FILEFORMAT = CSV
  PATTERN = '*.csv'
  FORMAT_OPTIONS ('databricks.connection' = 'my_sharepoint_conn', 'header' =
'true', 'inferSchema' = 'true')
  COPY_OPTIONS ('mergeSchema' = 'true');

# Ingest a single Excel file
COPY INTO sharepoint_excel_table
  FROM
    "https://mytenant.sharepoint.com/sites/Finance/Shared%20Documents/Monthly/Re
    Oct.xlsx"
  FILEFORMAT = EXCEL
  FORMAT_OPTIONS ('databricks.connection' = 'my_sharepoint_conn', 'headerRow
  '1')
  COPY_OPTIONS ('mergeSchema' = 'true');
```

Ingest SharePoint files in Lakeflow Spark Declarative Pipelines

NOTE

The SharePoint connector requires Databricks Runtime 17.3 or later. To use the connector, set `"CHANNEL" = "PREVIEW"` in your pipeline settings. For more information on previews, see [Pipeline properties reference](#).

The following examples show how to read SharePoint files using Auto Loader in Lakeflow Spark Declarative Pipelines.

Python SQL

Python

```
from pyspark import pipelines as dp

# Incrementally ingest new PDF files
@dp.table
def sharepoint_pdf_table():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "binaryFile")
            .option("databricks.connection", "my_sharepoint_conn")
            .option("pathGlobFilter", "*.pdf")
            .load("https://mytenant.sharepoint.com/sites/Marketing/Shared%20Document
    )

# Incrementally ingest CSV files with automatic schema inference and evolution
@dp.table
def sharepoint_csv_table():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("databricks.connection", "my_sharepoint_conn")
            .option("pathGlobFilter", "*.csv")
            .option("inferColumnTypes", True)
            .option("header", True)
            .load("https://mytenant.sharepoint.com/sites/Engineering/Data/IoT_Logs
    )

# Read a specific Excel file from SharePoint in a materialized view
@dp.table
def sharepoint_excel_table():
```

 Ask Genie

```

return (spark.read.format("excel")
    .option("databricks.connection", "my_sharepoint_conn")
    .option("headerRows", 1) # optional
    .option("inferColumnTypes", True) # optional
    .option("dataAddress", "Sheet1!A1:M20") # optional

    .load("https://mytenant.sharepoint.com/sites/Finance/Shared%20Documents/Mont
Oct.xlsx"))

```

Parse unstructured files

When ingesting unstructured files from SharePoint (such as PDFs, Word documents, or PowerPoint files) using the standard SharePoint connector with `binaryFile` format, the file contents are stored as raw binary data. To prepare these files for AI workloads—such as RAG, search, classification, or document understanding—you can parse the binary content into structured, queryable output using `ai_parse_document`.

The following example shows how to parse unstructured documents stored in a bronze Delta table named `documents`, adding a new column with parsed content:

SQL

```

CREATE TABLE documents AS
SELECT * FROM read_files(
    "https://mytenant.sharepoint.com/sites/Marketing/Shared%20Documents",
    `databricks.connection` => "my_sharepoint_conn",
    format => "binaryFile",
    pathGlobFilter => "*.pdf,docx",
    schemaEvolutionMode => "none"
);
SELECT *, ai_parse_document(content) AS parsed_content
FROM documents;

```

The `parsed_content` column contains extracted text, tables, layout information, and metadata that can be directly used for downstream AI pipelines.

Incremental parsing with Lakeflow Spark Declarative Pipelines

You can also use `ai_parse_document` within Lakeflow Spark Declarative Pipelines to enable incremental parsing. As new files stream in from SharePoint, they are automatically parsed as your pipeline updates.

For example, you can define a materialized view that continuously parses newly ingested documents:

SQL

```
CREATE OR REFRESH STREAMING TABLE sharepoint_documents_table
AS SELECT * FROM STREAM read_files(
  "https://mytenant.sharepoint.com/sites/Marketing/Shared%20Documents",
  format => "binaryFile",
  `databricks.connection` => "my_sharepoint_conn",
  pathGlobFilter => "*.pdf,docx");

CREATE OR REFRESH MATERIALIZED VIEW documents_parsed
AS
SELECT *, ai_parse_document(content) AS parsed_content
FROM sharepoint_documents_table;
```

This approach ensures that:

- Newly ingested SharePoint files are parsed automatically whenever the materialized view refreshes
- Parsed outputs stay in sync with incoming data
- Downstream AI pipelines always operate on up-to-date document representations

Learn more: See [ai_parse_document](#) for supported formats and advanced options.

Limitations

The standard SharePoint connector has the following limitations.

- **No multi-site ingestion:** You cannot ingest multiple sites with the same query. To ingest from two sites, you must write two separate queries.
- **Filtering:** You can use the `pathGlobFilter` option to filter files by name. Folder path-based filtering is not supported.

 Ask Genie

- **Unsupported formats:** SharePoint Lists and .aspx site pages are not supported. Only files in Document Libraries are supported.
- Writing back to a SharePoint server is not supported.
- Auto Loader `cleanSource` (deleting or archiving files at the source after ingestion) is not supported.

Next steps

- Learn about [Auto Loader](#) for advanced streaming ingestion patterns
- Explore [COPY INTO](#) for idempotent incremental loads
- Compare with [cloud object storage ingestion](#) patterns
- Set up [job scheduling](#) to automate your ingestion workflows
- Use [Lakeflow Spark Declarative Pipelines](#) to build end-to-end data pipelines with transformations

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 9, 2026**

Zerobus Ingest connector overview

Zerobus Ingest is a push-based ingestion API that writes data directly into Unity Catalog Delta tables. It is a serverless connector that automatically scales to handle incoming connections. It does not require configuring partitions or managing brokers.

With Zerobus Ingest, your "scaling strategy" is to open more connections. This streamlines ingestion workflows by eliminating the need for message bus infrastructure.

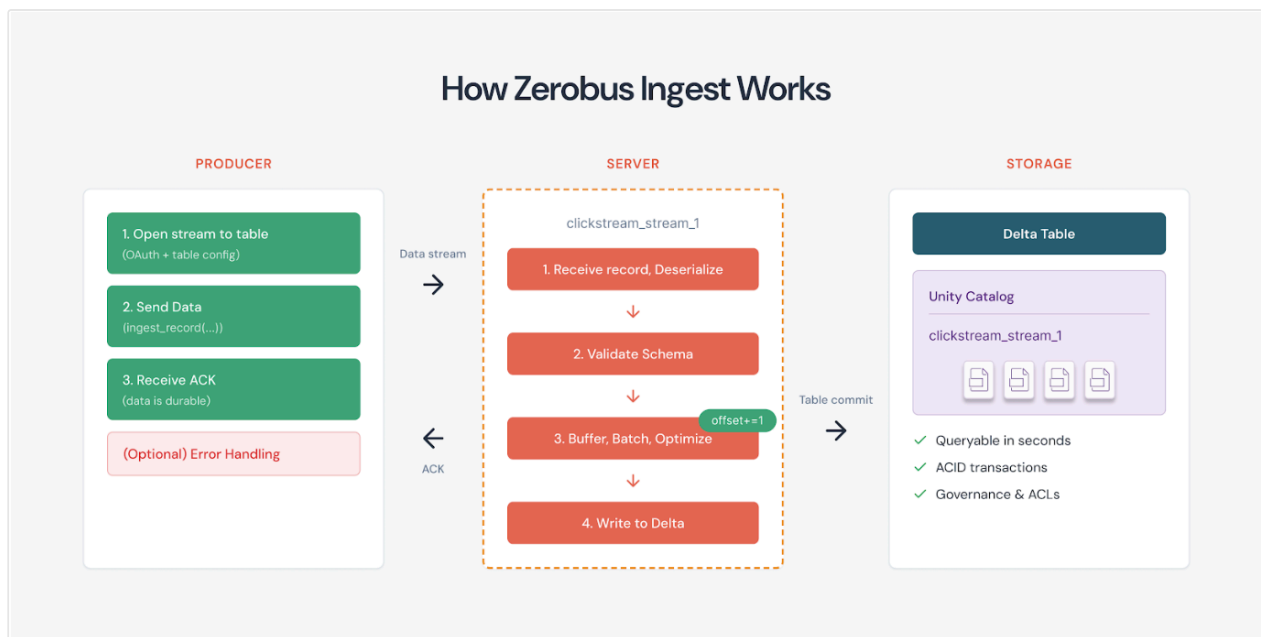
Any application that can integrate with Zerobus Ingest SDKs or communicate via supported APIs (gRPC, REST, and OpenTelemetry) can use Zerobus Ingest to push data efficiently into Delta tables.

The Zerobus Ingest buffers transmit data before adding it to a Delta table. This buffering creates an efficient and durable ingestion mechanism that supports a high volume of clients with variable throughput.

Once materialized in Delta format, the data becomes fully compatible with the comprehensive Databricks Data Intelligence Platform, enabling users to leverage familiar tools and capabilities for further data analysis and processing.

Concepts

A data producer first opens a stream to Zerobus Ingest API and specifies a target Delta table, constructs a message matching its schema, and then pushes the message via the opened stream. The service makes the data durable, acknowledges the client's message, and materializes the data in the Delta table in an optimized fashion.



Streams

A stream is a direct connection between your client and the Zerobus Ingest server. The SDKs use streams to facilitate long-lived, high-throughput connections.

- Streams are only used in the gRPC API with the SDKs.
- A stream ingests data to a single target table.
- Event delivery ordering is guaranteed on the per-stream level.
- Additional streams can be opened to write data to different tables.
- Additional streams can be opened to increase client throughput.
 - **Note:** If your client round-robins messages to different streams, ordering will no longer be guaranteed.

Server

The Zerobus Ingest service does not automatically create or manipulate tables. Users must create the table themselves. Tables and their schemas are the authoritative sources for the expectations of incoming data.

The Zerobus Ingest server accepts data sent to it by clients, validates that it fits the target table schema, and then writes it to the table. If the data fits in the table, it will be durable. An acknowledgment is then sent back to the client.

The service responsibilities include:

- Schema validation of the message to the table.
- Materializing the data in a timely manner in the target table.
- Sending an acknowledgement to the client that the data is durable.

Client

Client integration involves:

- Selecting a target table.
- Establishing a stream with the Zerobus Ingest service.
- Constructing a schema-compatible message.
- Sending the message.
- Managing message acknowledgements.
- Implementing recovery mechanisms in the case of client, stream, or server-side failures (e.g., connection issues, schema mismatches)

The Zerobus Ingest SDKs provide user-friendly methods to accomplish client integration, and the documentation offers examples of different development patterns. For custom integrations, the SDKs can serve as a reference for integration structure and recovery handling.

Get started with Zerobus Ingest

NOTE

If you have a client-side firewall, add the IP address used by Zerobus Ingest to your allowlist. To view IP addresses by region, see [IP addresses and domains for Databricks services and assets](#).

1. Get a Zerobus Ingest URL.
2. Create or identify the table you want to ingest data into.
3. Create a service principal and grant privileges to the table.
4. Connect a client or exporter to start sending data.

Choose the guide for your use case:

- **Ingest your own data:** Use the Zerobus Ingest SDKs or REST API with a schema you define. For full instructions, see [Use the Zerobus Ingest connector](#).
- **Ingest OpenTelemetry data:** Use standard OpenTelemetry SDKs or collectors to send traces, logs, and metrics into predefined table schemas. For full instructions, see [Ingest OpenTelemetry data with Zerobus Ingest](#).

Durable Fallback Location

If a breaking change is made to your target table after Zerobus Ingest makes your data durable before Zerobus Ingest has a chance to publish (push file to storage), the connector will make the data available in a separate folder within your table's storage location.

Data is persisted as Parquet files under the following path relative to the table root directory: `_zerobus/table_rejected_parquets/`

This directory is created directly under the table's physical root storage location, and all Parquet files generated by the service are written there. This ensures that all persisted data remains within the table's storage boundary and follows the same access controls and lifecycle policies as the table itself.

Cost

Charges for Zerobus Ingest will be billed against the "Jobs Serverless" SKU. Pricing is available on the [Lakeflow Connect pricing page](#).

Monitoring your usage

There are two methods for monitoring your Zerobus Ingest connector usage.

Zerobus Ingest system tables

You can monitor your usage via Zerobus Ingest system tables. See [Zerobus Ingest system table reference](#).

Billing system table

You can monitor your spend via the billable usage system table. See [Billable usage system table reference](#). Filter for Zerobus Ingest usage with:

- `billing_origin_product = 'LAKEFLOW_CONNECT'`
- `product_features.lakeflow_connect.zerobus_request_type = 'GRPC'` or `'HTTP'`

ⓘ BETA

OpenTelemetry (OTLP) ingestion is in [Beta](#) and is not billed at this time.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Sep 6, 2024**

Migrate data to Delta Lake

Databricks provides tools to simplify the migration of Parquet and Apache Iceberg data into Delta Lake.

Tool	When to use
<code>CONVERT TO DELTA SQL</code> command	This command performs a one-time, in place conversion of Parquet and Iceberg tables into external tables.
<code>CLONE</code>	<code>CLONE</code> support for Parquet and Iceberg extends <code>CONVERT TO DELTA</code> functionality to include incremental support for conversion. You can use <code>DEEP CLONE</code> to create Unity Catalog managed tables.

Databricks recommends storing data using Unity Catalog managed tables, but in-place conversion provides many of the same benefits without needing to fully rewrite all data. Databricks recommends using `CLONE` if the source system continues to receive updates during the migration.

Was this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **Apr 15, 2026**

Convert to Delta Lake

The `CONVERT TO DELTA` SQL command performs a one-time conversion for Parquet and Apache Iceberg tables to Delta Lake tables. For incremental conversion of Parquet or Iceberg tables to Delta Lake, see [Incrementally clone Parquet and Apache Iceberg tables to Delta Lake](#).

Unity Catalog supports the `CONVERT TO DELTA` SQL command for Parquet and Iceberg tables stored in external locations managed by Unity Catalog.

You can configure existing Parquet data files as external tables in Unity Catalog and then convert them to Delta Lake to unlock all features of the Databricks lakehouse.

For the technical documentation, see [CONVERT TO DELTA](#).

Converting a directory of Parquet or Iceberg files in an external location to Delta Lake

NOTE

- Converting Iceberg tables is supported in Databricks Runtime 10.4 LTS and above.
- Converting Iceberg metastore tables is not supported.
- Converting Iceberg tables that have experienced [partition evolution](#) is not supported.
- The following are limitations for converting Iceberg tables with partitions defined on truncated columns:
 - In Databricks Runtime 12.2 LTS and below, the only truncated type supported is `string`.

- In Databricks Runtime 13.3 LTS and above, you can work with truncated columns of types `string`, `long`, or `int`.
- Databricks does not support working with truncated columns of type `decimal`.

You can convert a directory of Parquet data files to a Delta Lake table as long as you have write access on the storage location. For information on configuring access with Unity Catalog, see [Connect to cloud object storage using Unity Catalog](#).

SQL

```
CONVERT TO DELTA parquet.`s3://my-bucket/parquet-data`;
```

```
CONVERT TO DELTA iceberg.`s3://my-bucket/iceberg-data`;
```

To load converted tables as external tables to Unity Catalog, you need the `CREATE EXTERNAL TABLE` permission on the external location.

NOTE

For Databricks Runtime 11.3 LTS and above, `CONVERT TO DELTA` automatically infers partitioning information for tables registered to the Hive metastore. You must provide partitioning information for Unity Catalog external tables.

Converting managed and external tables to Delta Lake on Unity Catalog

`CONVERT TO DELTA` syntax can only be used for creating Unity Catalog external tables. Use a `CTAS` statement to convert a legacy Hive metastore managed Parquet table directly to a managed Unity Catalog Delta Lake table, see [Upgrade a Hive table to a Unity Catalog managed table using CREATE TABLE AS SELECT](#).

To upgrade an external Parquet table to a Unity Catalog external table, see [Upgrade a schema or tables from the Hive metastore to Unity Catalog external tables using the upgrade wizard](#).

After you've registered an external Parquet table to Unity Catalog, you can convert it to an external Delta Lake table. You must provide partitioning information if the Parquet table is partitioned.

SQL

```
CONVERT TO DELTA catalog_name.database_name.table_name;
```

```
CONVERT TO DELTA catalog_name.database_name.table_name PARTITIONED BY  
(date_updated DATE);
```

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 15, 2026**

Incrementally clone Parquet and Apache Iceberg tables to Delta Lake

You can use Databricks clone functionality to incrementally convert data from Parquet or Apache Iceberg data sources to managed or external Delta tables.

Databricks clone for Parquet and Iceberg combines functionality used to [clone Delta tables](#) and [convert tables to Delta Lake](#). This article describes use cases and limitations for this feature and provides examples.

ⓘ PREVIEW

This feature is in [Public Preview](#).

ⓘ NOTE

This feature requires Databricks Runtime 11.3 LTS or above.

When to use clone for incremental ingestion of Parquet or Iceberg data

Databricks provides a number of options for [ingesting data into the lakehouse](#).

Databricks recommends using clone to ingest Parquet or Iceberg data in the following situations:

ⓘ NOTE

 Ask Genie

The term *source table* refers to the table and data files to be cloned, while the *target table* refers to the Delta table created or updated by the operation.

- You are performing a migration from Parquet or Iceberg to Delta Lake, but need to continue using source tables.
- You need to maintain an ingest-only sync between a target table and production source table that receives appends, updates, and deletes.
- You want to create an [ACID-compliant](#) snapshot of source data for reporting, machine learning, or batch ETL.

What is the syntax for clone?

Clone for Parquet and Iceberg uses the same basic syntax used to clone Delta tables, with support for shallow and deep clones. For more information, see [Clone types](#).

Databricks recommends using clone incrementally for most workloads. Clone support for Parquet and Iceberg uses SQL syntax.

NOTE

Clone for Parquet and Iceberg has different requirements and guarantees than either clone or convert to Delta. See [Requirements and limitations for cloning Parquet and Iceberg tables](#).

To deep clone a Parquet or Iceberg table using a file path, use the following syntax:

SQL

```
CREATE OR REPLACE TABLE <target-table-name> CLONE  
parquet.`/path/to/data`;
```

```
CREATE OR REPLACE TABLE <target-table-name> CLONE  
iceberg.`/path/to/data`;
```

To shallow clone a Parquet or Iceberg table using a file path, use the following syntax:

SQL

 Ask Genie

```
CREATE OR REPLACE TABLE <target-table-name> SHALLOW CLONE  
parquet.`/path/to/data`;
```

```
CREATE OR REPLACE TABLE <target-table-name> SHALLOW CLONE  
iceberg.`/path/to/data`;
```

You can also create deep or shallow clones for Parquet tables registered to the metastore, as shown in the following examples:

SQL

```
CREATE OR REPLACE TABLE <target-table-name> CLONE <source-table-name>;
```

```
CREATE OR REPLACE TABLE <target-table-name> SHALLOW CLONE <source-table-  
name>;
```

Requirements and limitations for cloning Parquet and Iceberg tables

Whether using deep or shallow clones, changes applied to the target table after the clone occurs cannot be synced back to the source table. Incremental syncing with clone is unidirectional, allowing changes to source tables to be automatically applied to target Delta tables.

The following additional limitations apply when using clone with Parquet and Iceberg tables:

- You must register Parquet tables with partitions to a catalog such as Unity Catalog or the legacy Hive metastore before cloning and using the table name to identify the source table. You cannot use path-based clone syntax for Parquet tables with partitions.
- You cannot clone Iceberg tables that have experienced partition evolution.
- The following are limitations for cloning Iceberg tables with partitions defined on truncated columns:
 - In Databricks Runtime 12.2 LTS and below, the only truncated column type supported is `string`.

- In Databricks Runtime 13.3 LTS and above, you can work with truncated columns of types `string`, `long`, or `int`.
- Databricks does not support working with truncated columns of type `decimal`.
- Incremental clone syncs the schema changes and properties from the source table. Any schema changes and data files written directly to the cloned table are overridden.
- Unity Catalog does not support shallow clones for Parquet or Iceberg tables.
- You cannot use glob patterns when defining a path.

NOTE

In Databricks Runtime 11.3 LTS, this operation does not collect file-level statistics. As such, target tables do not benefit from Delta Lake data skipping. File-level statistics are collected in Databricks Runtime 12.2 LTS and above.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback